

TỐI ƯU HÓA NÂNG CAO

Tối ưu hóa hàm mất mát Ridge

cho bài toán dự đoán lãi suất khoản vay

Bộ dữ liệu Lending Club, 2007–2018

Nhóm ...
Lớp Khoa học dữ liệu

Ngày 14 tháng 8 năm 2026

Tóm tắt

Báo cáo so sánh bốn thuật toán tối ưu hóa tự cài đặt, gradient descent đầy đủ, mini-batch SGD, gradient tăng tốc Nesterov và phương pháp Newton, mỗi thuật toán với cả bước cố định lẫn backtracking, trước hết trên hàm mất mát Ridge dựng từ 1 200 000 khoản vay Lending Club với $d = 116$ thuộc tính. Hàm mục tiêu lồi mạnh và có nghiệm đóng [1], nên giá trị tối ưu f^* tính được tới sai số máy và mọi biểu đồ hội tụ vẽ sai số $f(w_k) - f^*$ trên thang logarit.

Ở cấu hình tốt nhất của từng phương pháp, Newton đạt ngưỡng 10^{-6} sau một vòng lặp và khoảng 0,25 giây, gradient tăng tốc cần 75 vòng và 1,05 giây, backtracking cần 143 vòng và 3,14 giây, còn gradient descent với bước cố định tốt nhất cần 331 vòng và 4,75 giây. SGD nhanh nhất trên mỗi đơn vị thời gian nhưng dừng ở sai số $6,0 \cdot 10^{-4}$ vì bước hằng chỉ đưa dãy lặp vào một lân cận của nghiệm.

Vì hàm bậc hai cho Newton nghiệm sau một vòng lặp và cho backtracking không việc gì để làm, chương 8 chạy lại cả bốn thuật toán trên một hàm mục tiêu thứ hai dựng từ đúng bộ dữ liệu đó, mất mát Huber với cùng số hạng hiệu chỉnh. Newton ở đó cần 7 vòng lặp, và biến thể dùng lại Hessian, vốn tốt nhất trên Ridge, không còn hội tụ nếu thiếu line search.

Năm quan sát đi ngược dự đoán ban đầu và được giải thích trong báo cáo: bước $1,9/L$ nhanh gấp hơn hai lần bước tối ưu $2/(L + \mu)$ của lý thuyết, do sai số ban đầu dồn vào các hướng trị riêng lớn; backtracking thắng bước cố định trên cả hai trục đo chứ không chỉ trên trục vòng lặp; tốc độ giảm của bước SGD quyết định kết luận về quy tắc chọn bước, tới mức hai giá trị hợp lý cho hai kết luận ngược nhau; trên mất mát Huber, dùng lại Hessian với bước đầy đủ khóa dãy lặp vào một chu trình hai điểm thay vì chỉ làm chậm nó; và cũng trên hàm đó, backtracking lùi bước với hướng gradient nhưng không lùi lần nào với hướng Newton.

Mục lục

Tóm tắt	1
1 Bài toán và ba hằng số	3
1.1 Hàm mục tiêu	3
1.2 Ba hằng số chi phối tốc độ hội tụ	3
1.3 Cách đo sai số	4
2 Dữ liệu: rò rỉ và chuẩn hóa	5
2.1 Bộ dữ liệu và các cột phải loại	5

2.2	Một hướng suy biến do cách ghi dữ liệu	5
2.3	Chuẩn hóa cột và số điều kiện	5
2.4	Chọn hệ số hiệu chỉnh	7
3	Cài đặt và kiểm thử	9
3.1	Hai trục của bài toán	9
3.2	Điều kiện Armijo ở dạng tổng quát	9
3.3	Bốn phép kiểm tra	10
4	Chọn độ dài bước thế nào	11
4.1	Bước cố định và ngưỡng phân kỳ	11
4.2	Chênh lệch giữa $1,9/L$ và bước tối ưu lý thuyết	11
4.3	Dò bước mà không tính L	13
4.4	Backtracking	13
5	Quán tính: đổi bộ nhớ lấy số vòng lặp	16
5.1	Ba công thức momentum	16
5.2	Khởi động lại thích nghi	18
6	Ngẫu nhiên hóa: đổi độ chính xác lấy giá mỗi vòng	20
6.1	Kích thước lô	20
6.2	Bước khởi đầu	20
6.3	Quy tắc giảm bước, và một kết luận phụ thuộc tham số	23
7	Thông tin bậc hai: đổi giá mỗi vòng lấy số vòng lặp	26
7.1	Một vòng lặp, sai số bằng không	26
7.2	Giá của một vòng lặp	26
8	Khi hàm mục tiêu không còn là hàm bậc hai	29
8.1	Mất mát Huber trên cùng bộ dữ liệu	29
8.2	Newton mất đặc quyền một vòng	30
8.3	Backtracking có việc với hướng nào	31
8.4	Thứ hạng bốn thuật toán trên hàm phi tuyến	34
9	So sánh tổng hợp trên hai trục	37
9.1	Cấu hình tốt nhất của từng phương pháp	37
9.2	Ảnh hưởng của kích thước mẫu	37
9.3	Đối chiếu với tốc độ lý thuyết	40
10	So sánh với scikit-learn	41
10.1	Quy đổi hàm mục tiêu	41
10.2	Kết quả	41
11	Ảnh hưởng của hệ số hiệu chỉnh	44
12	Kết luận	47
	Tài liệu tham khảo	48

Chương 1

Bài toán và ba hằng số

1.1 Hàm mục tiêu

Bài toán hồi quy Ridge [2] trên n điểm dữ liệu và d thuộc tính là bài toán cực tiểu hóa không ràng buộc

$$\min_{w \in \mathbb{R}^d} f(w) = \frac{1}{2n} \|Xw - y\|^2 + \frac{\lambda}{2} \|w\|^2, \quad (1.1)$$

với $X \in \mathbb{R}^{n \times d}$ là ma trận thiết kế đã chuẩn hóa, $y \in \mathbb{R}^n$ là vector lãi suất đã trừ trung bình, và $\lambda > 0$ là hệ số hiệu chỉnh. Hệ số chặn không xuất hiện trong (1.1) vì đã bị khử bằng cách trừ trung bình khỏi cả X và y ; giữ nó lại mà không phạt cũng cho cùng nghiệm, nhưng làm công thức gradient và Hessian dài thêm mà không đổi nội dung.

Hệ số $\frac{1}{2n}$ thay cho $\frac{1}{2}$ có một lý do cụ thể cho phần so sánh sau này. Không có $\frac{1}{n}$ thì hằng số Lipschitz tăng tuyến tính theo số mẫu, nên độ dài bước dò được trên mẫu 200 000 điểm sẽ không dùng lại được trên mẫu 1 200 000 điểm, và mọi cấu hình phải quét lại từ đầu ở quy mô lớn.

Gradient và Hessian của (1.1) là

$$\nabla f(w) = \frac{1}{n} X^\top (Xw - y) + \lambda w, \quad \nabla^2 f(w) = \frac{1}{n} X^\top X + \lambda I. \quad (1.2)$$

Hessian trong (1.2) không chứa w , vì f là hàm bậc hai. Tính chất này chi phối toàn bộ chương 7, vì khai triển Taylor bậc hai của f không còn số hạng dư và bước Newton xuất phát từ $w_0 = 0$ do đó trùng đúng với nghiệm đóng

$$w^* = \left(\frac{1}{n} X^\top X + \lambda I \right)^{-1} \frac{1}{n} X^\top y. \quad (1.3)$$

1.2 Ba hằng số chi phối tốc độ hội tụ

Ký hiệu $\lambda_{\max}$ và $\lambda_{\min}$ là trị riêng lớn nhất và nhỏ nhất của ma trận Gram $\frac{1}{n} X^\top X$. Hằng số Lipschitz của gradient, hệ số lỗi mạnh và số điều kiện là

$$L = \lambda_{\max} + \lambda, \quad \mu = \lambda_{\min} + \lambda, \quad \kappa = \frac{L}{\mu}. \quad (1.4)$$

Ba đại lượng này đo được chứ không phải giả định, vì $d = 116$ đủ nhỏ để tính trọn phổ của ma trận Gram trong dưới một giây. Bảng 1.1 ghi giá trị ở hai quy mô mẫu.

Ba hằng số ở hai quy mô lệch nhau dưới 0,6 phần trăm, cụ thể 0,59 phần trăm với L , 0,11 phần trăm với μ và 0,48 phần trăm với κ . Hệ số $\frac{1}{n}$ trong (1.1) khử ảnh hưởng của kích thước mẫu lên phổ, nên hai bài toán tuy khác nhau về số điểm lại có cùng

Bảng 1.1: Ba hằng số của bài toán ở hai quy mô mẫu, với $\lambda = 0,031623$ và $d = 116$ cho cả hai.

Quy mô	n	L	μ	κ	f^*
Quét tham số	200 000	9,1156	0,034073	267,5	6,151114
Toàn phần	1 200 000	9,0620	0,034036	266,2	6,142806

độ khó về mặt tối ưu hóa. Nhờ vậy toàn bộ phần quét lưới tham số chạy được trên mẫu nhỏ, và chỉ cấu hình tốt nhất mới phải chạy lại ở quy mô đầy đủ.

Nghiệm đóng (1.3) giải bằng phân rã Cholesky cho $\|\nabla f(w^*)\| = 7,4 \cdot 10^{-13}$, tức ở mức sai số máy. Nhờ con số đó, báo cáo đo được sai số tuyệt đối $f(w_k) - f^*$ của từng thuật toán thay vì chỉ so sánh các đường cong với nhau. Nếu đổi sang hàm mất mát không có nghiệm đóng, chẳng hạn Huber hay Lasso, thì f^* phải giải bằng số trước khi vẽ được bất cứ đường nào; chương 8 giải nó bằng một lần chạy Newton riêng tới sai số máy.

1.3 Cách đo sai số

Vẽ $f(w_k) - f^*$ bằng phép trừ trực tiếp có một giới hạn số học đáng kể. Hai số gần bằng nhau khi trừ mất hết chữ số có nghĩa ở phần thập, nên với $f^* \approx 6,15$ thì mọi giá trị dưới khoảng $10^{-16}f^*$ đều là nhiễu làm tròn. Vì f là hàm bậc hai và $\nabla f(w^*) = 0$, khai triển Taylor không còn số hạng dư và đẳng thức

$$f(w) - f^* = \frac{1}{2}(w - w^*)^\top \nabla^2 f(w - w^*) \quad (1.5)$$

đúng chính xác chứ không phải xấp xỉ. Báo cáo dùng vế phải của (1.5).

Trên bài toán thử với $n = 400$ và $d = 12$, tại $\|w - w^*\| \approx 10^{-8}$ phép trừ trực tiếp chỉ còn đúng ba chữ số, và tại 10^{-12} nó trả về $-1,4 \cdot 10^{-17}$, tức một giá trị âm không có ý nghĩa, trong khi (1.5) cho $3,7 \cdot 10^{-24}$. Trục tung của mọi hình hội tụ trong báo cáo do đó xuống được sâu hơn tám bậc so với cách vẽ thông thường. Cách này chỉ dùng được vì f bậc hai; với hàm tổng quát thì (1.5) chỉ còn là xấp xỉ bậc hai quanh w^* .

Chương 2

Dữ liệu: rò rỉ và chuẩn hóa

2.1 Bộ dữ liệu và các cột phải loại

Bộ dữ liệu Lending Club gồm 2 260 668 khoản vay được duyệt trong giai đoạn 2007–2018 với 151 cột, sau khi loại 33 dòng thiếu biến mục tiêu. Biến cần dự đoán là lãi suất `int_rate`, nằm trong khoảng từ 5,31 tới 30,99 phần trăm với độ lệch chuẩn 4,83.

Cột `sub_grade` phải loại trước mọi bước khác. Hồi quy `int_rate` theo riêng `sub_grade` cho $R^2 = 0,9554$ trên toàn bộ dữ liệu, và $R^2 = 0,9834$ khi xét riêng các khoản vay phát hành năm 2016, còn `grade` cho $R^2 = 0,9088$. Lending Club ấn định lãi suất từ một bảng tra theo bậc tín dụng, nên hai cột này chính là biến mục tiêu mang tên khác; con số trong phạm vi một năm cao hơn vì bảng tra ấy cố định trong năm và thay đổi giữa các năm. Cột `installment` cũng bị loại, vì nó tính ra từ `loan_amnt`, kỳ hạn và chính `int_rate` bằng công thức trả góp. Sau khi loại cả nhóm cột ghi nhận sau giải ngân, mô hình đạt $R^2 \approx 0,486$ trên tập kiểm tra. Chênh lệch giữa 0,955 và 0,486 là thước đo trực tiếp của phần rò rỉ.

Với riêng phần tối ưu hóa, rò rỉ không làm sai một phép tính nào, vì hàm mục tiêu vẫn lồi mạnh và mọi thuật toán vẫn hội tụ về cùng một w^* . Nó chỉ làm hỏng phần đối chiếu chất lượng dự báo ở chương 11, nơi RMSE trên tập kiểm tra được dùng để cân nhắc hệ số hiệu chỉnh.

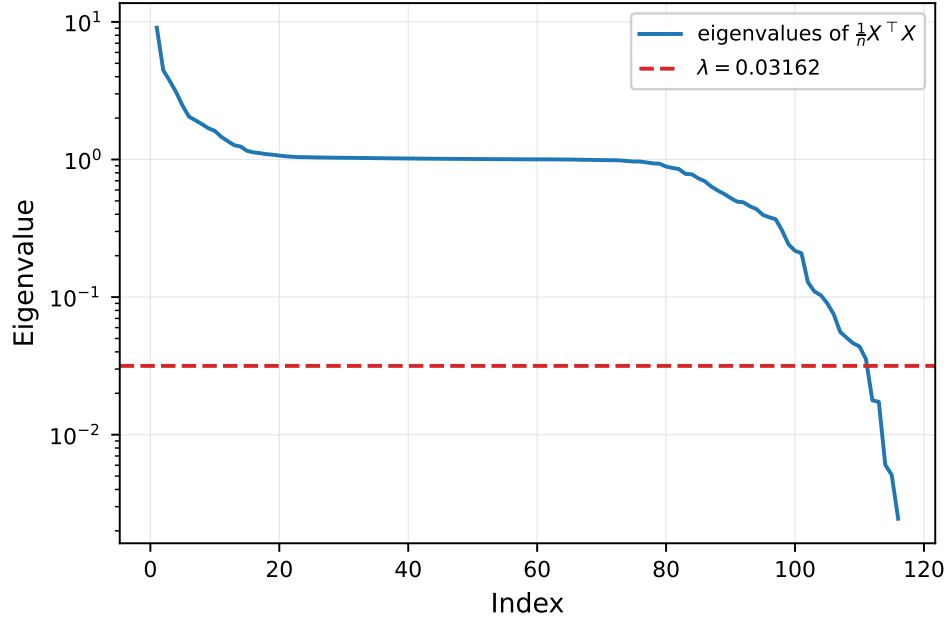
2.2 Một hướng suy biến do cách ghi dữ liệu

Lần dựng ma trận thiết kế đầu tiên cho $d = 117$ với trị riêng nhỏ nhất của ma trận Gram bằng $1,07 \cdot 10^{-7}$, kéo theo $\mu = \lambda$ đúng tới sáu chữ số. Vector riêng ứng với trị riêng đó có đúng hai hệ số khác không, bằng +0,7071 tại `fico_range_low` và -0,7071 tại `fico_range_high`, tức đúng hiệu hai cột chia cho $\sqrt{2}$.

Truy lại dữ liệu gốc thì `fico_range_high` bằng `fico_range_low` cộng 4 ở 2 260 227 trên 2 260 668 dòng, với hệ số tương quan 0,99999991. Hai cột lệch nhau đúng một hằng số nên trùng nhau hoàn toàn sau khi chuẩn hóa, và ma trận Gram nhận một hướng gần suy biến không mang thông tin nào. Đã loại `fico_range_high` cùng nhóm với `funded_amnt` và `funded_amnt_inv`, và d giảm còn 116. Sau khi loại, trị riêng nhỏ nhất là $2,45 \cdot 10^{-3}$, nên $\mu = 0,0341$ vẫn do $\lambda = 0,0316$ chi phối tới 93 phần trăm nhưng không còn suy biến. Hình 2.1 vẽ phổ trị riêng cùng với sà đo λ đặt.

2.3 Chuẩn hóa cột và số điều kiện

Bảng 2.1 so sánh ba cách mã hóa cùng một tập dòng: giữ nguyên thang gốc, chỉ trừ trung bình, và chuẩn hóa đầy đủ về trung bình 0 độ lệch chuẩn 1.



Hình 2.1: Phổ trị riêng của ma trận Gram $\frac{1}{n}X^T X$ và sần do hệ số hiệu chỉnh đặt. Mọi trị riêng nằm dưới đường ngang đều bị λ thay thế trong công thức $\mu = \lambda_{\min} + \lambda$.

Bảng 2.1: Ảnh hưởng của cách xử lý cột lên số điều kiện, đo với $\lambda = 0,031623$ và $n = 200\,000$. Cột cuối là số vòng lặp gradient descent cần để đạt $f - f^* < 10^{-6}$ với bước $1/L$.

Cách mã hóa	L	μ	κ	Số vòng lặp
Thô	$1,225 \cdot 10^{11}$	0,031628	$3,87 \cdot 10^{12}$	không đạt
Chỉ trừ trung bình	$6,039 \cdot 10^{10}$	0,031628	$1,91 \cdot 10^{12}$	không đạt
Chuẩn hóa đầy đủ	9,1156	0,034073	267,5	630

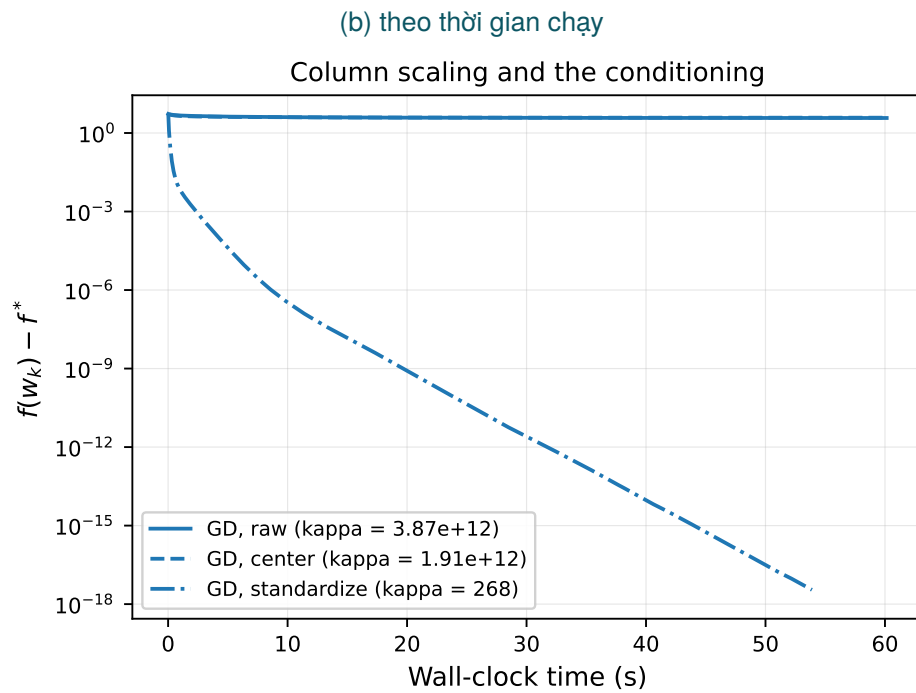
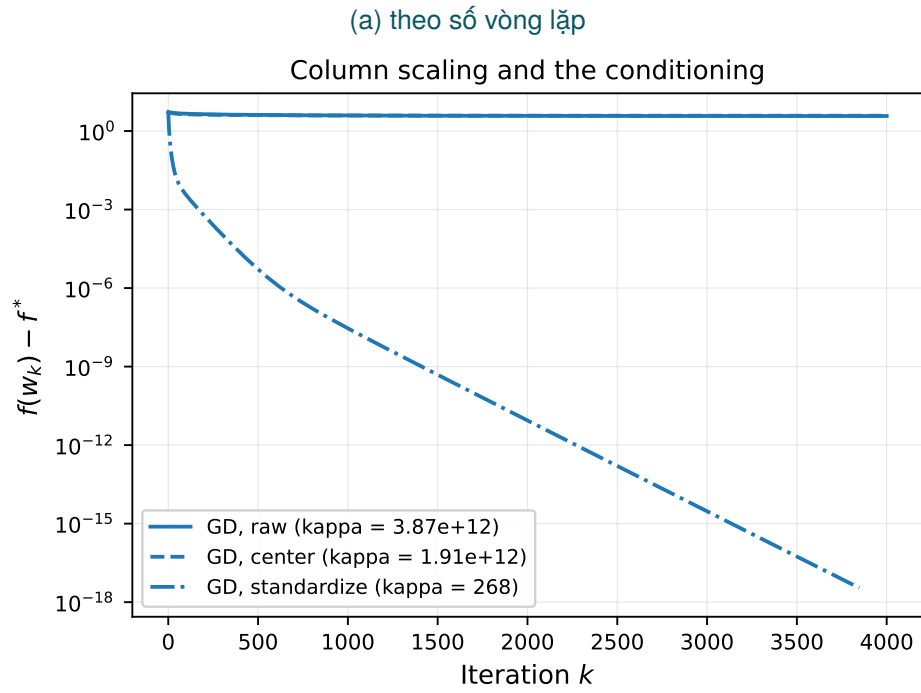
Số điều kiện giảm $1,45 \cdot 10^{10}$ lần khi chuyển từ thang gốc sang thang chuẩn hóa, nhưng phần đóng góp của hai thao tác rất chênh lệch. Trừ trung bình chỉ hạ κ đúng hai lần, còn chia độ lệch chuẩn hạ tiếp $7 \cdot 10^9$ lần. Các cột đứng ở những thang khác nhau tới sáu bậc, `tot_cur_bal` cỡ 10^6 bên cạnh `dti` cỡ hàng chục, nên cột có đơn vị lớn nhất chi phối trị riêng lớn nhất; trừ đi trung bình không đựng gì tới chênh lệch thang đo đó.

Ở hai biến thể chưa chia thang, μ bằng đúng λ tới năm chữ số, tức ma trận Gram suy biến về mặt số học khi đặt cạnh L cỡ 10^{11} . Hình 2.2 cho thấy hệ quả: sau 4000 vòng lặp, gradient descent mới hạ được sai số từ 5,53 xuống 3,74, tức chưa tới một phần ba. Chuẩn hóa vì thế không phải một thói quen tiền xử lý mà là một can thiệp trực tiếp lên độ khó của bài toán tối ưu hóa. Kết luận này mất hiệu lực nếu mọi cột vốn đã cùng thang, khi đó phép chia không còn gì để sửa.

2.4 Chọn hệ số hiệu chỉnh

Hệ số λ chọn bằng cross-validation 5 fold trên tập huấn luyện, quét trên lưới logarit từ 10^{-6} tới 10^2 . Đường cong sai số phẳng trên nhiều bậc λ , nên lấy điểm cực tiểu là lựa chọn tùy tiện. Cực tiểu của một đường phẳng rơi vào chỗ nhiễu đặt nó, và ở đây nó rơi vào $\lambda = 0,001778$, tức gần đầu nhỏ nhất của lưới và do đó ứng với κ lớn nhất.

Báo cáo dùng quy tắc một sai số chuẩn [3], tức chọn λ lớn nhất còn nằm trong một sai số chuẩn của giá trị tốt nhất. Kết quả là $\lambda = 0,031623$, lớn hơn 18 lần so với điểm cực tiểu, đổi lại sai số cross-validation tăng từ 12,0430 lên 12,0607, tức 0,15 phần trăm. Chương 11 cho thấy khoản đánh đổi này mua được gì về phía tối ưu hóa. Quy tắc sẽ không phù hợp nếu đường cong cross-validation có cực tiểu nhọn, khi đó dịch λ ra xa cực tiểu phải trả giá thật về chất lượng dự báo.



Hình 2.2: Gradient descent với bước $1/L$ trên ba cách mã hóa cột. Hai đường trên cùng gần như nằm ngang trong suốt 4000 vòng lặp.

Chương 3

Cài đặt và kiểm thử

3.1 Hai trục của bài toán

Đề bài yêu cầu bốn thuật toán, mỗi thuật toán hai cơ chế chọn bước, tức một ma trận bốn nhân hai. Mã nguồn tách đúng hai trục đó thành hai loại đối tượng rời nhau. Trục thứ nhất trả lời câu hỏi đi từ điểm nào theo hướng nào, gồm bốn lớp *SteepestDescent*, *MiniBatch*, *Nesterov* và *NewtonStep*. Trục thứ hai trả lời câu hỏi đi bao xa, gồm ba lớp *Fixed*, *Armijo* và *Decay*. Một hàm lặp duy nhất ghép hai trục lại, giữ đồng hồ, ghi log và kiểm tra điều kiện dừng.

Cách tách này có ba hệ quả thực tế. Tám cấu hình bắt buộc trở thành tích Descartes của hai danh sách, nên câu hỏi đã cài đủ hay chưa trả lời được bằng một vòng lặp trong bộ kiểm thử thay vì bằng cách đọc lại mã. Quy tắc dừng đồng hồ trước khi ghi log chỉ phải kiểm tra ở một chỗ thay vì bốn. Và điểm xuất phát của bước được tách khỏi dãy lặp hiện tại, vì *Nesterov* tính gradient tại điểm ngoại suy y_k chứ không tại w_k ; gộp hai khái niệm này là lỗi thường gặp khi cài *Nesterov*.

Cách tách không phủ hết mọi thuật toán. Adam co giãn theo từng tọa độ nên không có độ dài bước vô hướng, và nếu bổ sung thì phần co giãn phải nằm trong lớp hướng đi, làm nhòe ranh giới hai trục.

3.2 Điều kiện Armijo ở dạng tổng quát

Thuật toán 1 là backtracking line search theo điều kiện giảm đủ của Armijo [4].

Thuật toán 1 Backtracking line search (Armijo)

Require: descent direction p at y , parameters $c \in (0, 1)$, $\rho \in (0, 1)$, $t_0 > 0$

```
1:  $t \leftarrow t_0$ 
2: while  $f(y + tp) > f(y) + ct \nabla f(y)^\top p$  do
3:    $t \leftarrow \rho t$ 
4: end while
5: return  $t$ 
```

Điều kiện dừng viết theo tích vô hướng $\nabla f(y)^\top p$ chứ không theo $-\|\nabla f(y)\|^2$. Hai dạng trùng nhau khi $p = -\nabla f(y)$, nhưng chỉ dạng tổng quát còn đúng với hướng Newton, nơi $p = -(\nabla^2 f)^{-1} \nabla f$ và do đó $\nabla f^\top p = -\nabla f^\top (\nabla^2 f)^{-1} \nabla f < 0$. Vì lớp *Armijo* nhận hướng từ bên ngoài và không biết ai tạo ra nó, nó buộc phải dùng dạng tổng quát.

3.3 Bốn phép kiểm tra

Trước khi chạy lưới tham số, mã nguồn qua bốn phép kiểm tra. So sánh gradient giải tích với sai phân trung tâm cho sai số tương đối $1,2 \cdot 10^{-9}$, và so sánh Hessian với sai phân của gradient cho $1,8 \cdot 10^{-9}$; cả hai đều dưới ngưỡng 10^{-6} mà bậc ε^2 của sai phân trung tâm cho phép kỳ vọng. Nghiệm đóng cho $\|\nabla f(w^*)\| = 2,7 \cdot 10^{-15}$ trên bài toán thử. Phép kiểm cuối chạy toàn bộ tám cấu hình trên bài toán nhỏ có nghiệm biết trước và xác nhận cả tám đều về cùng một w^* với sai số dưới 10^{-8} .

Một chi tiết cài đặt cần nêu vì nó từng gây lỗi. Công thức momentum $\beta = (\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$ chỉ đúng khi $t = 1/L$, nên khi ghép với backtracking phải tính lại theo bước thật bằng $\beta = (1 - \sqrt{t\mu})/(1 + \sqrt{t\mu})$. Lớp `Nesterov` nhận đúng giá trị t mà line search vừa chọn qua hàm `accept`, nên hai đại lượng không có đường nào để lệch nhau. Vì t chỉ biết sau khi line search chạy xong còn β phải có trước để dựng y_k , cài đặt dùng bước của vòng trước và lấy $1/L$ cho vòng đầu; với bước cố định thì hai giá trị trùng nhau.

Chương 4

Chọn độ dài bước thế nào

4.1 Bước cố định và ngưỡng phân kỳ

Lý thuyết cho biết gradient descent với bước hằng hội tụ khi $0 < t < 2/L$, và với hàm bậc hai lồi mạnh thì bước tối ưu là $t = 2/(L + \mu)$ [5]. Bảng 4.1 ghi kết quả quét tám giá trị quanh ngưỡng đó.

Ngưỡng $2/L$ hiện ra đúng như lý thuyết mô tả. Với $t = 2,1/L$, hệ số co dọc hướng dốc nhất là $|1 - tL| = 1,1 > 1$, nên thành phần sai số theo hướng đó nở ra mỗi vòng và dãy lặp phân kỳ sau 94 vòng. Với $t = 2/L$ đúng bằng ngưỡng, hệ số bằng 1 nên thành phần đó dao động mà không tắt, và sai số dừng ở $9,4 \cdot 10^{-2}$. Hình 4.1 vẽ cả ba trường hợp.

4.2 Chênh lệch giữa $1,9/L$ và bước tối ưu lý thuyết

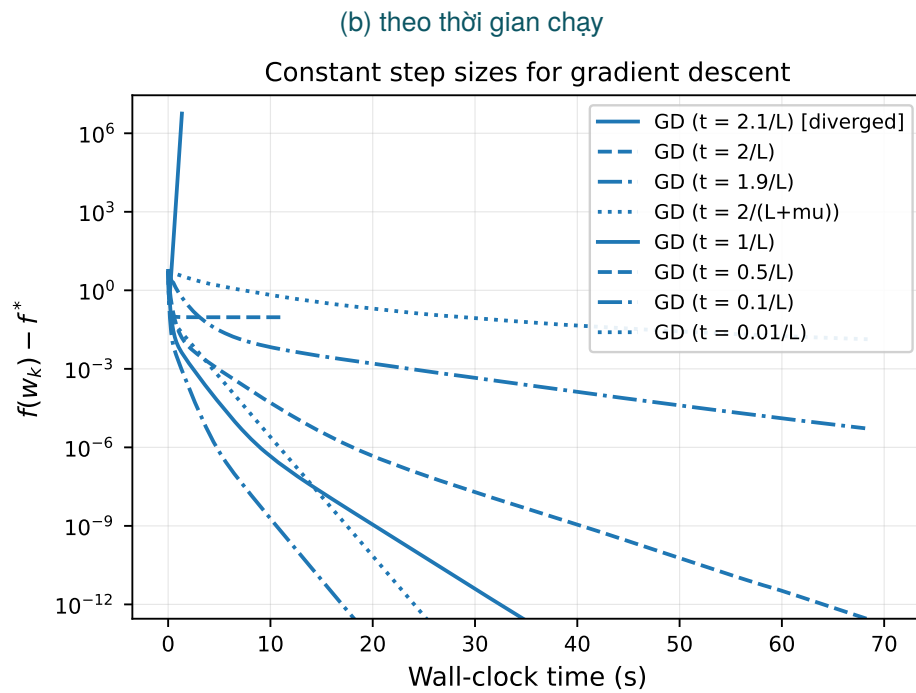
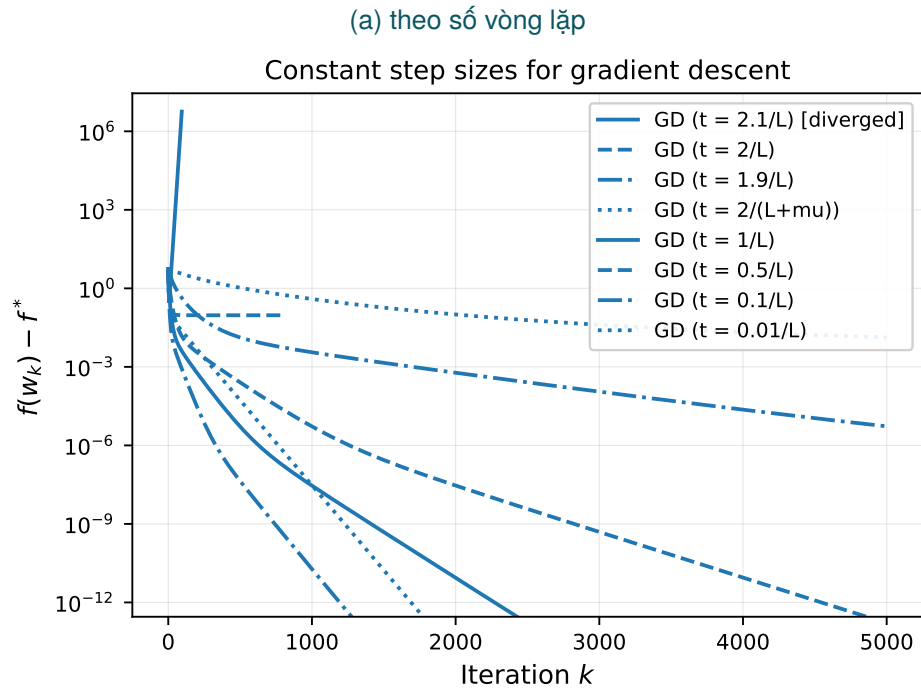
Bước $2/(L + \mu)$ cần 766 vòng lặp trong khi bước $1,9/L$ chỉ cần 331, tức chậm hơn 2,3 lần dù lý thuyết gọi nó là bước tối ưu. Lý thuyết không sai; nó trả lời một câu hỏi khác.

Cận $((\kappa - 1)/(\kappa + 1))^{2k}$ là cận cực tiểu hóa trường hợp xấu nhất trên mọi điểm khởi tạo, nên bước tối ưu cân bằng hai đầu phổ, tức là với $t = 2/(L + \mu)$ thì $|1 - tL| = 0,9926$ và $|1 - t\mu| = 0,99255$, gần bằng nhau. Bước $1,9/L$ hy sinh một chút ở đầu nhỏ, $|1 - t\mu| = 0,99290$, để đổi lấy $|1 - tL| = 0,9000$ ở đầu lớn.

Điểm khởi tạo $w_0 = 0$ không phải trường hợp xấu nhất. Phân tích sai số ban đầu theo cơ sở riêng của Hessian cho thấy các hướng có trị riêng lớn hơn 1 mang 80,5 phần trăm của $f(w_0) - f^*$, trong khi các hướng có trị riêng nhỏ hơn 0,1 chỉ mang 0,3 phần trăm, và riêng hướng ứng với trị riêng 3,76 chiếm 35,9 phần trăm. Chính dạng

Bảng 4.1: Gradient descent với bước cố định, $n = 200\,000$, $\kappa = 267,5$. Cột giữa là số vòng lặp cần để đạt $f - f^* < 10^{-6}$.

Bước	Số vòng lặp	Thời gian (s)	Trạng thái
$t = 2,1/L$	–	–	phân kỳ ở vòng 94
$t = 2/L$	–	–	đình trệ ở $9,4 \cdot 10^{-2}$
$t = 1,9/L$	331	4,75	hội tụ
$t = 2/(L + \mu)$	766	10,87	hội tụ
$t = 1/L$	630	8,98	hội tụ
$t = 0,5/L$	1261	18,04	chạm giới hạn vòng lặp
$t = 0,1/L$	–	–	không đạt sau 5000 vòng



Hình 4.1: Gradient descent với các bước cố định khác nhau. Đường phân kỳ ứng với $t = 2,1/L$ được giữ lại trong hình chứ không loại bỏ.

Bảng 4.2: Backtracking line search, sáu cấu hình tốt nhất trong 18 cấu hình đã quét. Cột cuối là số lần đánh giá hàm mục tiêu trung bình mỗi vòng lặp.

c	ρ, t_0	Số vòng lặp	Thời gian (s)	Lần đánh giá f
0,3	$\rho = 0,5, t_0 = 1$	143	3,14	4,94
0,1	$\rho = 0,5, t_0 = 1$	149	3,23	4,04
10^{-4}	$\rho = 0,5, t_0 = 1$	167	3,27	3,95
0,3	$\rho = 0,5, t_0 = 10/L$	190	4,34	4,17
0,3	$\rho = 0,8, t_0 = 1$	312	9,62	7,77
0,3	$\rho = 0,9, t_0 = 1$	–	–	12,4

toàn phương (1.5) gây ra sự chênh lệch đó. Viết trong cơ sở riêng thì

$$f(w_k) - f^* = \frac{1}{2} \sum_{i=1}^d \lambda_i c_i^2 (1 - t\lambda_i)^{2k}, \quad (4.1)$$

nên mỗi hướng được cân theo đúng trị riêng của nó, và các hướng dốc đóng góp nhiều gấp bội vào sai số cần triệt tiêu.

Mô hình (4.1) dự đoán được con số chứ không chỉ chiều hướng: nó cho sai số $9,80 \cdot 10^{-7}$ tại vòng 331 với bước $1,9/L$, và $9,95 \cdot 10^{-7}$ tại vòng 766 với bước $2/(L + \mu)$, khớp với ngưỡng 10^{-6} ở đúng hai vòng lặp quan sát được. Kết luận đảo chiều khi cần độ chính xác rất cao: lúc đó mọi hướng dốc đã tắt và tốc độ do hướng ứng với μ quyết định, nơi $2/(L + \mu)$ nhỉnh hơn. Với ngưỡng 10^{-6} mà báo cáo dùng, giai đoạn đó chưa tới.

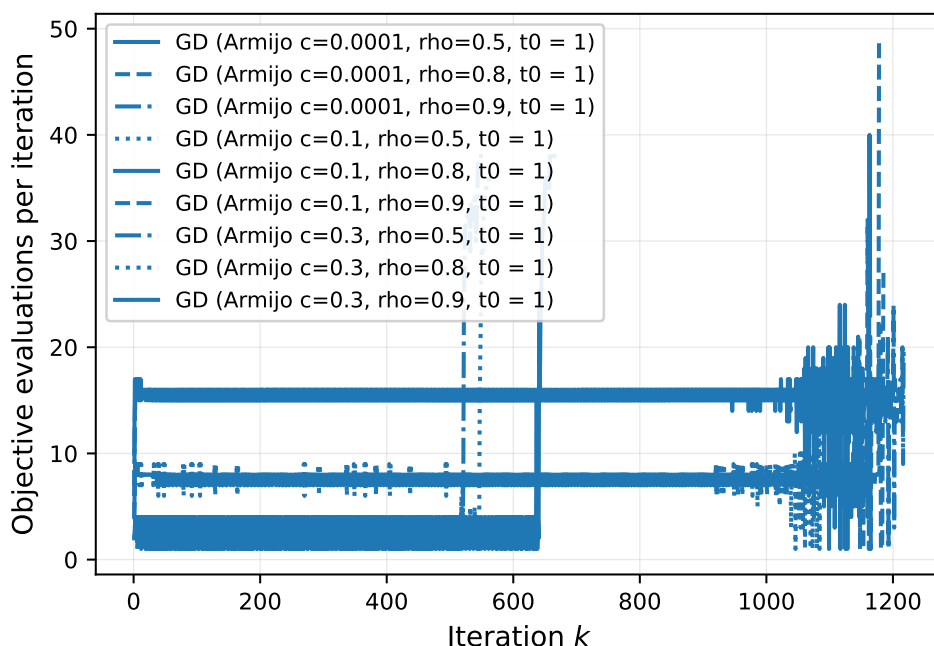
4.3 Dò bước mà không tính L

Đề bài gợi ý thử các bước 10^{-3} , 10^{-2} , 10^{-1} , 1 và 10. Trong lưới đó chỉ $t = 0,1$ hội tụ, sau 691 vòng lặp; $t = 0,01$ không đạt ngưỡng sau 5000 vòng, còn $t = 1$ phân kỳ sau 5 vòng và $t = 10$ sau 2 vòng. Vùng hội tụ là khoảng $(0; 0,219)$, nên lưới cách nhau một bậc chỉ có đúng một điểm rơi vào đó. Tính L và μ tốn dưới một giây, còn quét năm giá trị tốn 195 giây và vẫn không tìm được bước gần tối ưu.

4.4 Backtracking

Bảng 4.2 ghi sáu cấu hình backtracking tốt nhất trong lưới 18 cấu hình đã quét.

Backtracking thẳng bước cố định trên cả hai trục đo, chứ không phải chỉ thẳng về số vòng lặp và thua về thời gian như thường được mô tả: 143 vòng và 3,14 giây so với 331 vòng và 4,75 giây của bước cố định tốt nhất, dù mỗi vòng tốn thêm 4,94 lần đánh giá hàm mục tiêu. Cơ chế nằm ở chỗ điều kiện Armijo không so với L toàn cục mà so với thương Rayleigh tại chỗ: bước được chấp nhận thỏa $t \leq 2(1 - c)\|g\|^2 / (g^\top \nabla^2 f g)$, nên khi gradient nghiêng về các hướng phẳng thì mẫu số nhỏ và line search nhận bước lớn hơn $2/L$ rất nhiều. Bước cố định không làm được điều đó vì nó phải an toàn cho hướng dốc nhất ở mọi vòng lặp. Kết luận đảo chiều khi mỗi lần đánh giá f đắt



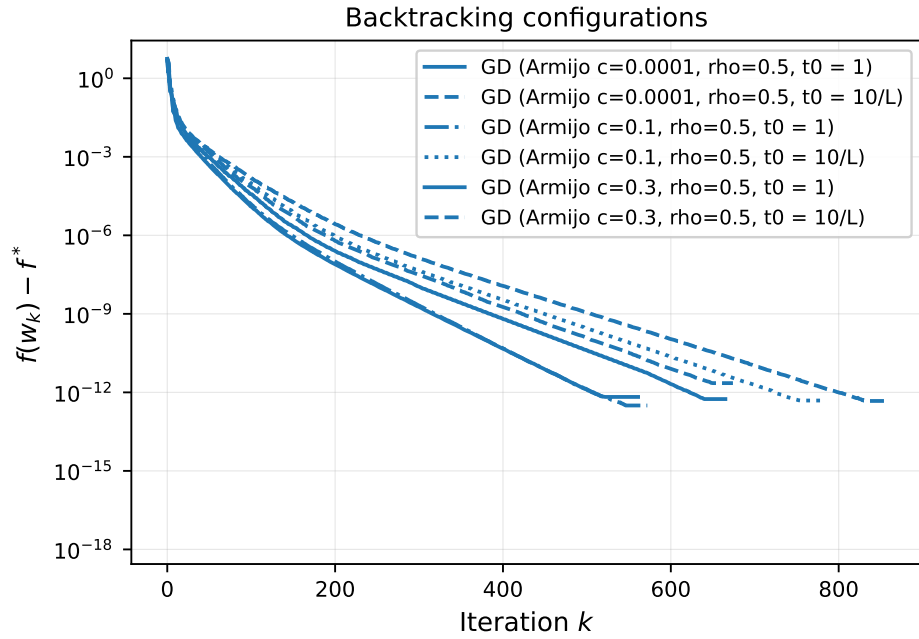
Hình 4.2: Số lần đánh giá hàm mục tiêu mỗi vòng lặp, cho các cấu hình backtracking với $t_0 = 1$.

ngang một lần tính gradient và κ nhỏ, khi đó phần chi phí thêm không còn được bù lại.

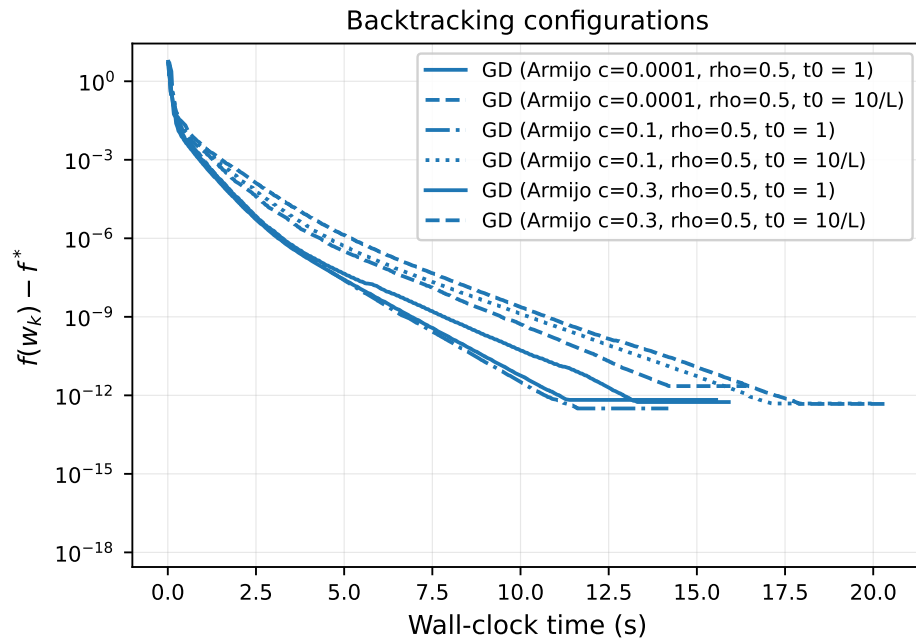
Hai tham số của line search không tương đương nhau về mức ảnh hưởng. Toàn bộ sáu cấu hình $\rho = 0,5$ đứng trên toàn bộ mười hai cấu hình $\rho = 0,8$ và $\rho = 0,9$, trong khi c đổi từ 10^{-4} lên $0,3$, tức 3000 lần, chỉ làm số vòng lặp nhích từ 167 xuống 143. Tham số ρ quyết định độ thô của lưới bước thử. Đi từ t_0 xuống cùng một bước, $\rho = 0,9$ cần gấp khoảng bảy lần số lần thử so với $\rho = 0,5$, và mỗi lần thử tốn một lần tính f .

Hình 4.3 vẽ các cấu hình này theo hai trục đo, còn hình 4.2 vẽ số lần đánh giá hàm theo vòng lặp và cho thấy chi phí không phân bố đều. Ở giai đoạn đầu, line search nhận bước ngay ở lần thử thứ nhất hoặc thứ hai. Về cuối, khi $f - f^*$ chạm sàn phân giải số học của f , mức giảm thật nhỏ hơn nhiều làm tròn nên mọi bước thử đều trượt điều kiện, và số lần đánh giá vọt lên hàng chục. Phần đuôi đó là chi phí chết, và tiêu chí dừng theo định trệ cắt được nó.

(a) theo số vòng lặp



(b) theo thời gian chạy



Hình 4.3: Backtracking với các cấu hình c và ρ khác nhau.

Chương 5

Quán tính: đổi bộ nhớ lấy số vòng lặp

Ba chương tiếp theo đặt bốn phương pháp vào cùng một khung đánh đổi giữa số vòng lặp và giá của mỗi vòng. Chương này xét cách rẻ nhất: giữ thêm một vector trong bộ nhớ mà không tăng chi phí mỗi vòng.

5.1 Ba công thức momentum

Gradient tăng tốc Nesterov cập nhật theo

$$y_k = w_k + \beta_k(w_k - w_{k-1}), \quad w_{k+1} = y_k - t \nabla f(y_k), \quad (5.1)$$

Cập nhật (5.1) vẫn dùng đúng một lần tính gradient mỗi vòng như gradient descent, chỉ thêm một vector w_{k-1} trong bộ nhớ. Bảng 5.1 so sánh ba cách chọn β_k với cùng bước $t = 1/L$.

Quán tính đổi một vector bộ nhớ lấy hơn mười lần số vòng lặp: 286 vòng và 4,01 giây so với 3000 vòng và 42,13 giây của gradient descent trên cùng bước. Vì giá mỗi vòng của hai phương pháp như nhau, tỷ số thời gian bằng đúng tỷ số số vòng lặp. Khoảng cách này thu hẹp khi κ nhỏ, vì $\sqrt{\kappa}$ và κ tiến lại gần nhau; chương 11 đo cụ thể.

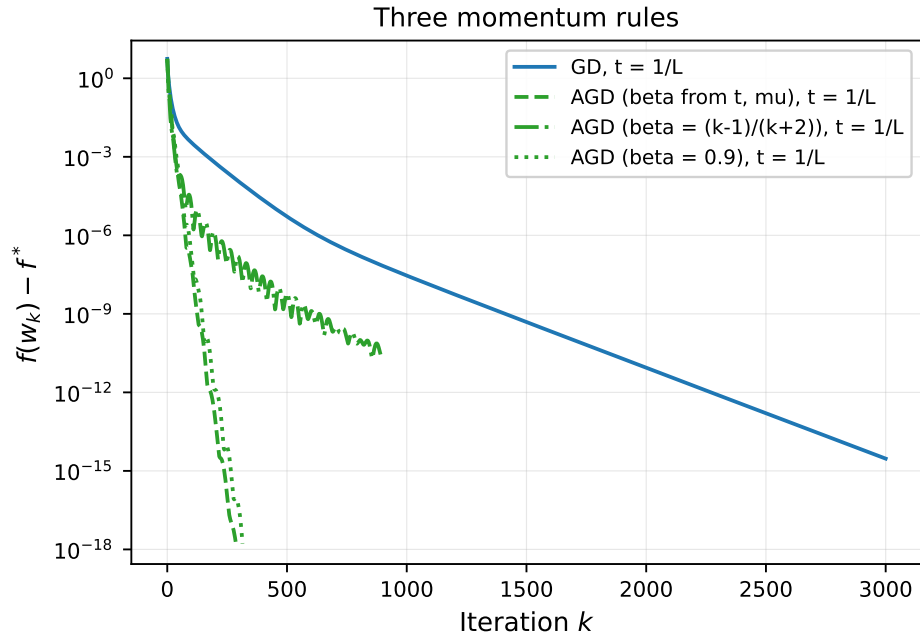
Hằng số $\beta = 0,9$ mà đề bài gợi ý cần 314 vòng, chỉ chậm hơn 10 phần trăm so với công thức lý thuyết. Không nên đọc con số đó thành một kết luận chung: với $\kappa = 267,5$ thì $(\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1) = 0,885$, tình cờ sát 0,9, và công thức momentum vốn không nhảy quanh cực trị. Với bài toán có κ lệch xa, chẳng hạn $\kappa = 10$ ứng với $\beta^* = 0,52$, hằng số 0,9 sẽ sai rõ.

Công thức $\beta_k = (k - 1)/(k + 2)$ đình trệ ở sai số $2,1 \cdot 10^{-11}$ sau 894 vòng. Dãy này tiến tới 1 nên momentum tích lũy vượt quá mức cần thiết và sinh dao động tuần hoàn thấy rõ trên hình 5.1. Công thức được thiết kế cho trường hợp không giả định biết μ , nên trên hàm lồi mạnh đã biết μ thì nó không có lý do để dùng.

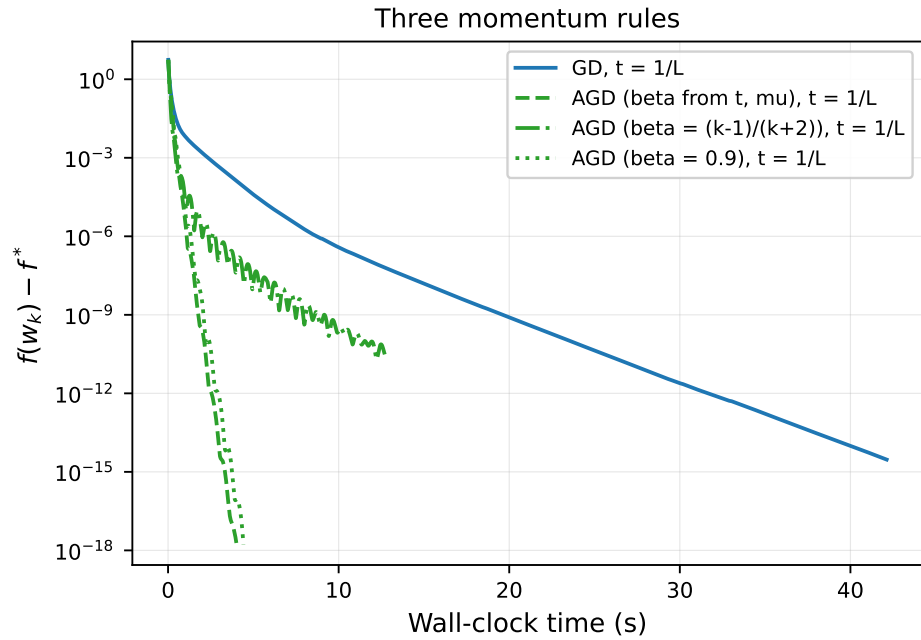
Bảng 5.1: Ba công thức momentum, cùng bước $t = 1/L$, so với gradient descent.

Công thức	Số vòng lặp	Thời gian (s)	Trạng thái
Gradient descent	3000	42,13	chưa hội tụ
β theo t và μ	286	4,01	hội tụ
$\beta = 0,9$	314	4,42	hội tụ
$\beta_k = (k - 1)/(k + 2)$	894	12,78	đình trệ ở $2,1 \cdot 10^{-11}$

(a) theo số vòng lặp



(b) theo thời gian chạy



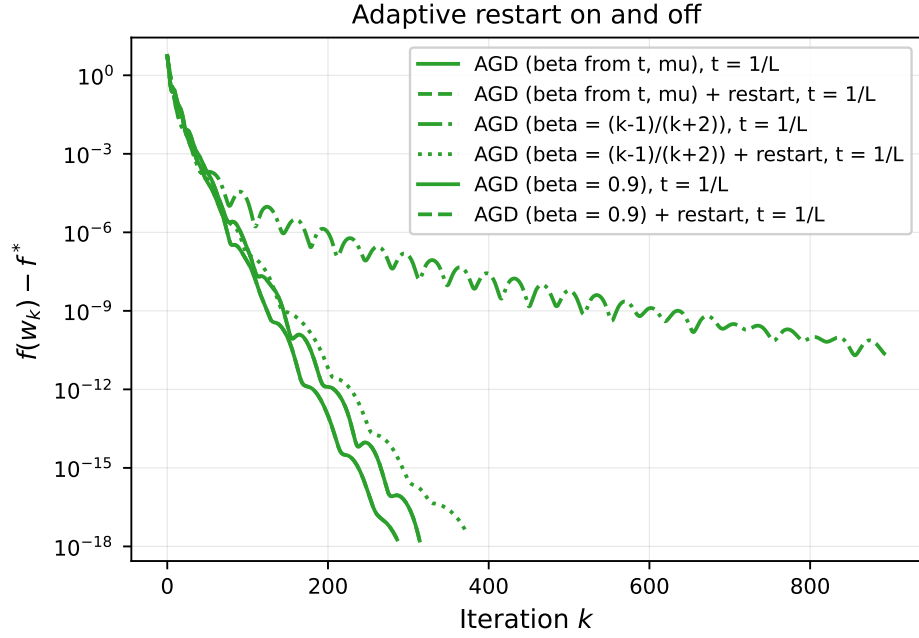
Hình 5.1: Ba công thức momentum so với gradient descent, cùng bước $t = 1/L$.

5.2 Khởi động lại thích nghi

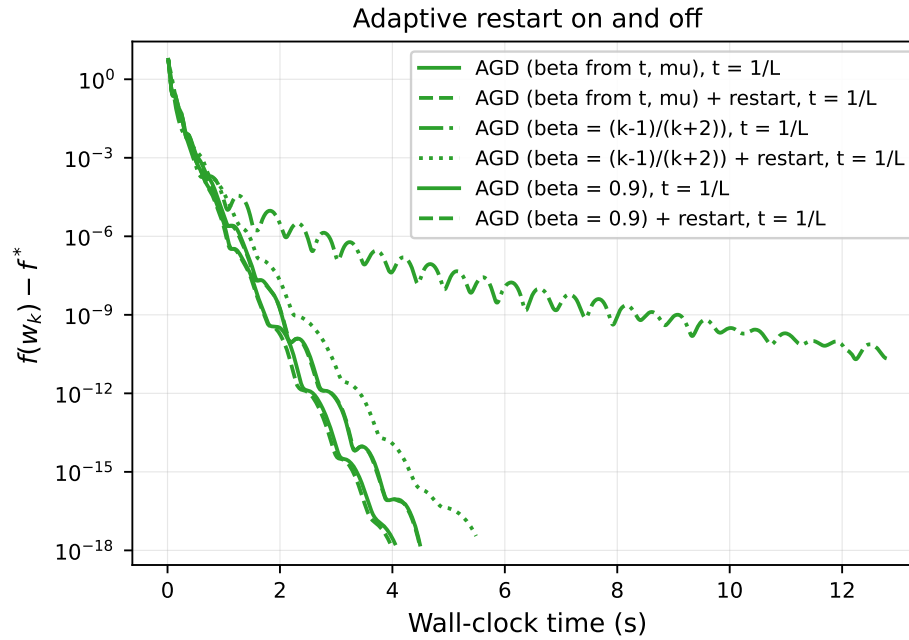
Kỹ thuật khởi động lại thích nghi [6] đặt bộ đếm momentum về 0 khi phát hiện $\nabla f(y_k)^\top (w_{k+1} - w_k) > 0$, tức khi quán tính đang đẩy ngược chiều gradient. Ba công thức ở bảng 5.1 phản ứng rất khác nhau. Với β_k tăng dần, số vòng lặp để đạt 10^{-6} giảm từ 143 xuống 89 và trạng thái chuyển từ đình trệ sang hội tụ, còn với β tính theo μ và với $\beta = 0,9$ thì kết quả có và không khởi động lại trùng nhau tới từng chữ số.

Hai kết quả trùng nhau vì điều kiện khởi động lại không kích hoạt lần nào khi β đã đặt đúng theo μ : momentum không bao giờ vượt đà nên không có gì để sửa. Khởi động lại vì thế là cơ chế bù cho việc không biết μ , chứ không phải một cải tiến chung. Khi μ không ước lượng được, nó trở thành bắt buộc. Hình 5.2 vẽ cả sáu lần chạy.

(a) theo số vòng lặp



(b) theo thời gian chạy



Hình 5.2: Ba công thức momentum, có và không có khởi động lại thích nghi.

Chương 6

Ngẫu nhiên hóa: đổi độ chính xác lấy giá mỗi vòng

6.1 Kích thước lô

Mini-batch SGD [7, 8] thay gradient đầy đủ bằng ước lượng trên một lô ngẫu nhiên $\mathcal{B}$ với $|\mathcal{B}| = B$:

$$g_k = \frac{1}{B} \sum_{i \in \mathcal{B}} (x_i^\top w_k - y_i) x_i + \lambda w_k, \quad w_{k+1} = w_k - \eta_k g_k. \quad (6.1)$$

Chi phí mỗi vòng của (6.1) giảm n/B lần, nên trục số vòng lặp không so sánh công bằng được SGD với các phương pháp đầy đủ; hình 6.1 vì thế vẽ thêm theo số epoch.

Hằng số Lipschitz của mất mát một mẫu lớn hơn L rất nhiều, vì L là trung bình còn nó là cực đại. Độ dài bước an toàn cho lô kích thước B do đó lấy theo

$$L_B = L + \frac{L_{\max} - L}{B}, \quad L_{\max} = \max_i \|x_i\|^2 + \lambda. \quad (6.2)$$

Với bước $1/L_B$ riêng cho từng B , mọi kích thước lô đều dừng quanh sai số $5 \cdot 10^{-4}$ sau 40 epoch, nhưng chi phí rất khác nhau: $B = 32$ tốn 250 000 vòng lặp và 4,16 giây, còn $B = 2048$ tốn 3880 vòng và 2,17 giây. Kích thước lô vì thế đổi giá mỗi vòng chứ không đổi mức sàn, do bước đặt theo $1/L_B$ đã bù trừ giữa phương sai của gradient ước lượng và số bước thực hiện được.

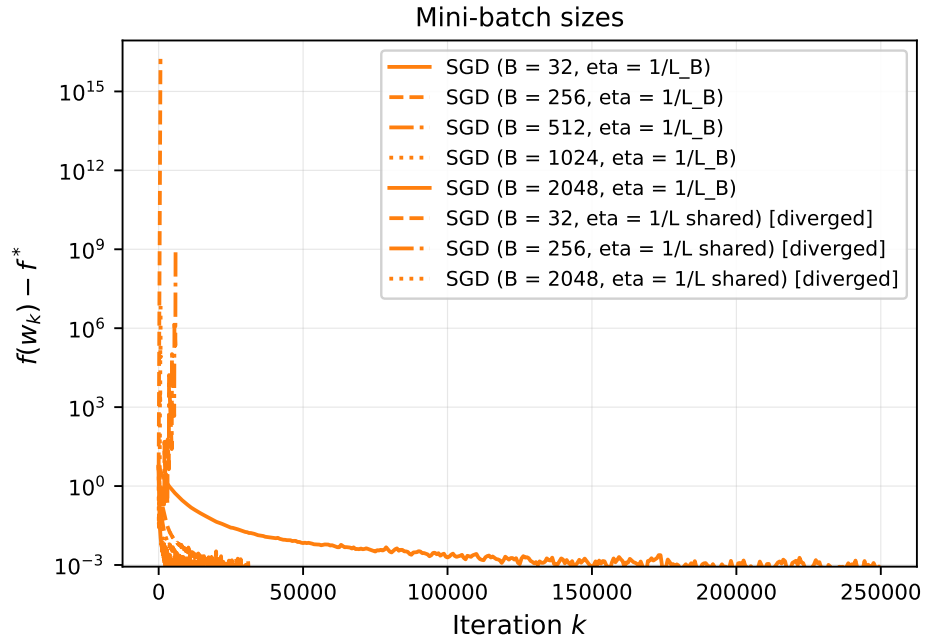
Dùng chung một bước $1/L$ cho mọi kích thước lô thì cả ba giá trị $B = 32$, $B = 256$ và $B = 2048$ đều phân kỳ. Đo trên bộ dữ liệu này, $L_{\max} = 200\,056$ so với $L = 9,12$, tức gấp 21 947 lần, nên ngay cả $B = 2048$ cũng chỉ hạ L_B xuống 106,8, vẫn lớn hơn L mười lần.

Con số $L_{\max}$ bị đúng một dòng dữ liệu kéo lên. Trung vị của $\|x_i\|^2$ là 82,2 và phân vị 99 là 560,9, trong khi cực đại là 200 056; tọa độ lớn nhất của dòng đó nằm ở cột chỉ dấu `addr_state=other` với giá trị 447 độ lệch chuẩn. Chuẩn hóa một cột chỉ dấu có tỷ lệ rất nhỏ biến mỗi giá trị 1 thành một đỉnh nhọn, nên $L_{\max}$ trong (6.2) ở đây phản ánh cách mã hóa nhiều hơn phản ánh dữ liệu.

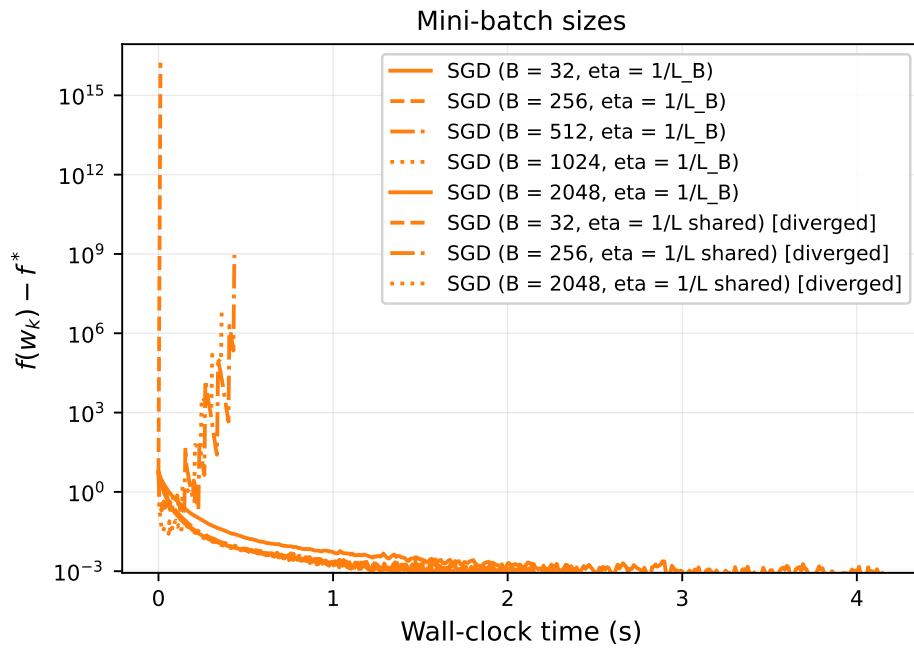
6.2 Bước khởi đầu

Dù L_B bị ngoại lai kéo lệch, bước $1/L_B = 0,001265$ tại $B = 256$ vẫn nằm đúng vùng tốt nhất đo được bằng thực nghiệm. Quét η_0 trên lưới logarit cho sai số cuối $4,21 \cdot 10^{-4}$ tại $\eta_0 = 10^{-3}$, $4,89 \cdot 10^{-3}$ tại $3 \cdot 10^{-4}$, rồi hồng hẩn ở $3 \cdot 10^{-3}$ với sai số $1,03 \cdot 10^3$ và phân kỳ từ 10^{-2} trở lên. Cận lý thuyết do đó vẫn đáng tính: nó là chặn trên an toàn, và ở bài này nó chặt. Hình 6.2 vẽ toàn bộ lưới.

(a) theo số vòng lặp

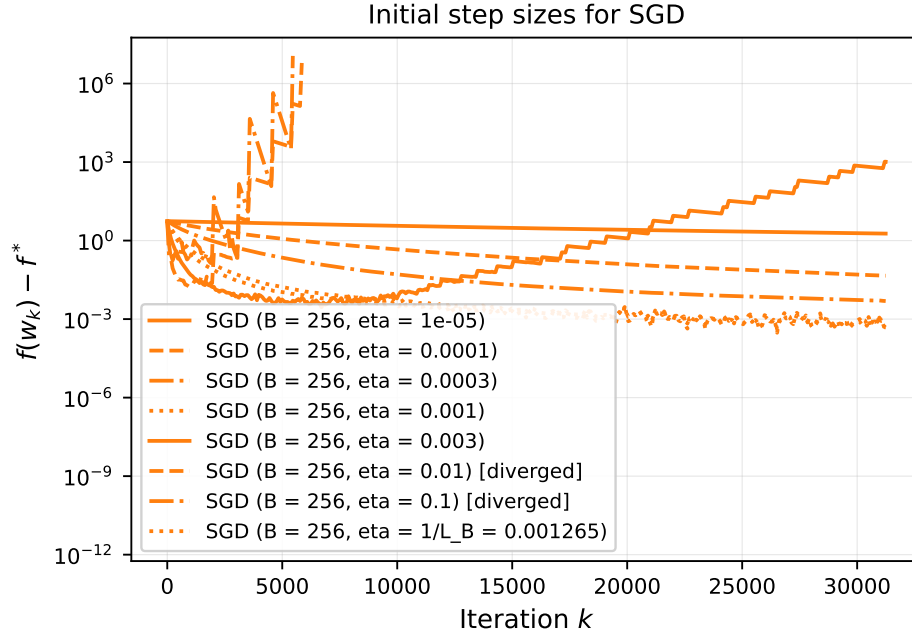


(b) theo thời gian chạy

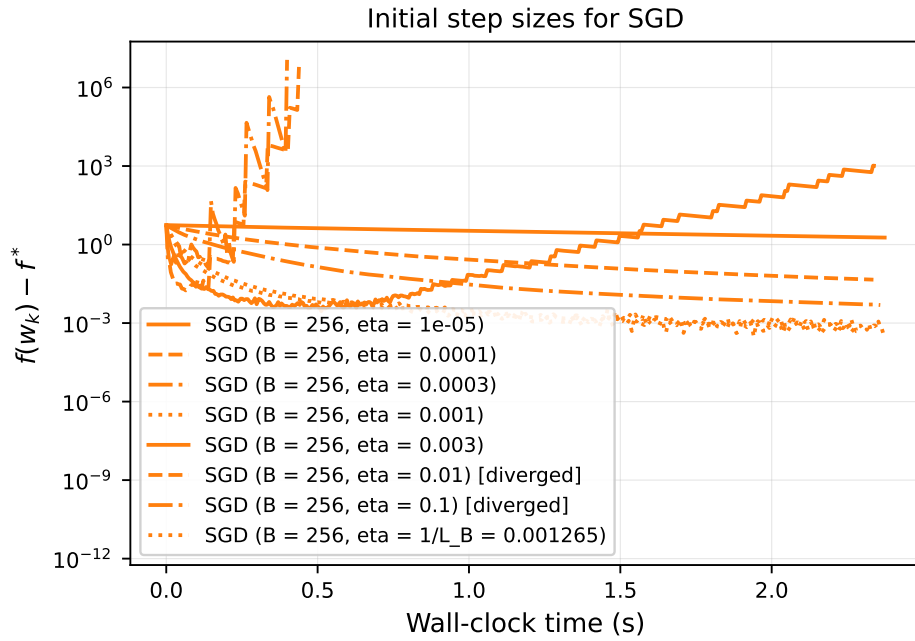


Hình 6.1: SGD với các kích thước lô khác nhau. Nhóm đường phân kỳ ứng với việc dùng chung bước $1/L$ cho mọi B .

(a) theo số vòng lặp



(b) theo thời gian chạy



Hình 6.2: SGD với các bước khởi đầu khác nhau, $B = 256$.

Bảng 6.1: Bốn quy tắc chọn bước cho SGD với $B = 256$, sai số sau 200 epoch, trung vị của 5 seed. Hai dòng *inverse* khác nhau ở tốc độ giảm.

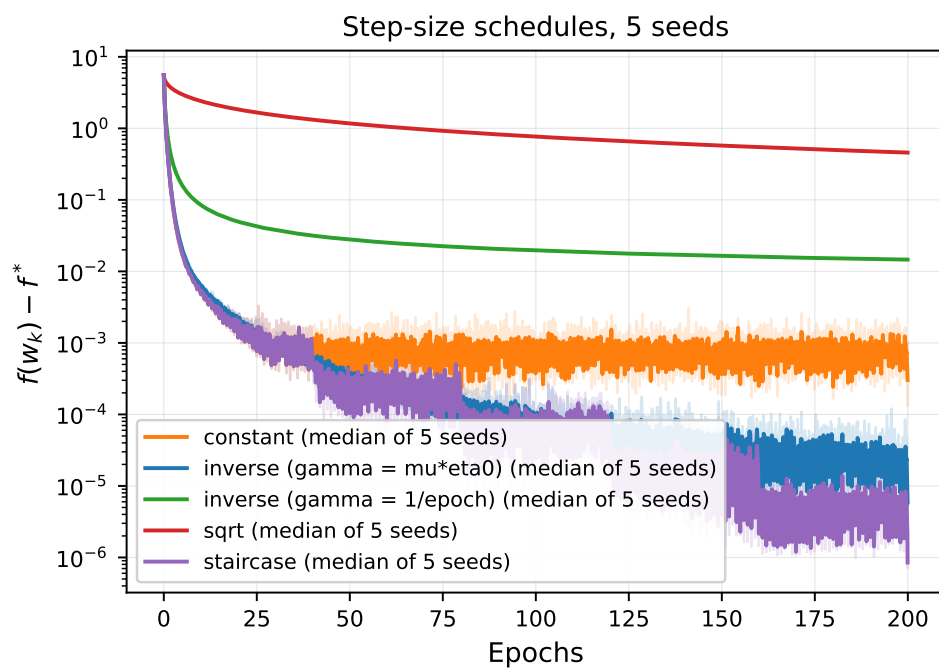
Quy tắc	η_k	$f - f^*$
Bậc thang, giảm nửa mỗi 40 epoch	$\eta_0 2^{-\lfloor k/P \rfloor}$	$8,49 \cdot 10^{-7}$
Nghịch đảo, $\gamma = \mu\eta_0$	$\eta_0 / (1 + \gamma k)$	$6,94 \cdot 10^{-6}$
Hằng số	η_0	$1,06 \cdot 10^{-3}$
Nghịch đảo, $\gamma = 1/\text{epoch}$	$\eta_0 / (1 + \gamma k)$	$1,46 \cdot 10^{-2}$
Căn bậc hai	$\eta_0 / \sqrt{k + 1}$	$4,58 \cdot 10^{-1}$

6.3 Quy tắc giảm bước, và một kết luận phụ thuộc tham số

SGD với bước hằng không hội tụ về w^* mà dao động trong một lân cận bán kính cỡ $O(\eta\sigma^2/\mu)$, với σ^2 là phương sai của gradient ngẫu nhiên [8]. Trên thang logarit, hiện tượng này hiện ra thành một đường nằm ngang, và nó là kết quả đúng cần giải thích chứ không phải lỗi cài đặt. Bảng 6.1 so sánh bốn quy tắc chọn bước sau 200 epoch.

Quy tắc giảm dần thẳng bước hằng khoảng ba bậc, nhưng chỉ khi tốc độ giảm đặt theo μ . Hai dòng nghịch đảo trong bảng 6.1 khác nhau đúng một tham số và cho hai kết luận ngược nhau: với $\gamma = 1/\text{epoch}$, một giá trị trông tự nhiên, bước tụt xuống $3 \cdot 10^{-6}$ ở epoch 400 và dấy đóng băng ở sai số $1,46 \cdot 10^{-2}$, cao hơn cả bước hằng; với $\gamma = \mu\eta_0$ theo lý thuyết cho hàm lồi mạnh, tức nhỏ hơn 30 lần, sai số xuống $6,94 \cdot 10^{-6}$. Ngân sách cũng quyết định kết luận: ở 40 epoch bước hằng vẫn đang dẫn, và chỉ từ khoảng 100 epoch trở đi thứ hạng mới đảo.

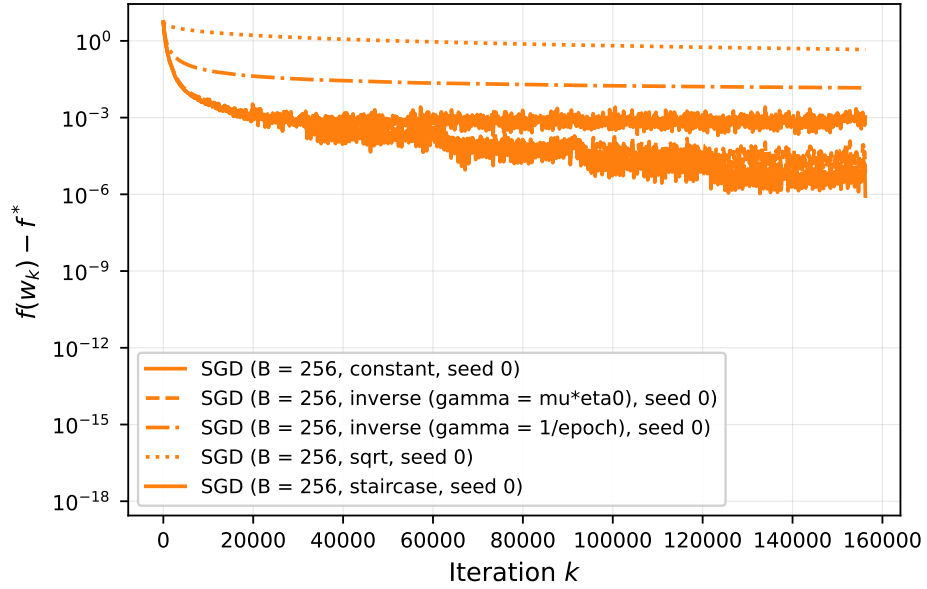
Bước hằng còn khác ba quy tắc kia ở mức phân tán giữa các lần chạy. Sai số cuối của nó trải từ $2,59 \cdot 10^{-4}$ tới $9,64 \cdot 10^{-4}$ qua 5 seed, tức chênh 3,7 lần, trong khi ba quy tắc giảm dần cho kết quả gần trùng nhau giữa các seed. Điểm dừng của bước hằng nằm trong một lân cận ngẫu nhiên nên phụ thuộc vào lô cuối cùng, còn quy tắc giảm dần đã co dấy lặp về gần một điểm tắt định. Hình 6.3 vẽ trung vị kèm dải min-max qua 5 seed, còn hình 6.4 vẽ riêng seed 0 theo hai trục đo.



Hình 6.3: Bốn quy tắc chọn bước, trung vị và dải min-max qua 5 seed.

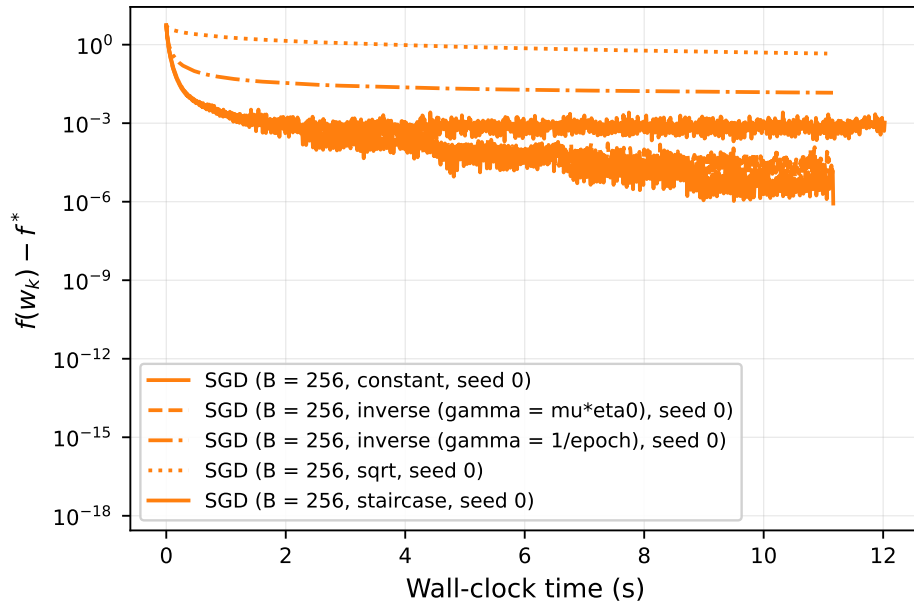
(a) theo số vòng lặp

Step-size schedules for SGD



(b) theo thời gian chạy

Step-size schedules for SGD



Hình 6.4: Bốn quy tắc chọn bước cho SGD, seed 0.

Chương 7

Thông tin bậc hai: đổi giá mỗi vòng lấy số vòng lặp

7.1 Một vòng lặp, sai số bằng không

Phương pháp Newton giải hệ $\nabla^2 f(w_k) p_k = -\nabla f(w_k)$ rồi cập nhật $w_{k+1} = w_k + t p_k$. Hệ được giải bằng phân rã Cholesky [9] chứ không bằng cách tính nghịch đảo tường minh, vì nghịch đảo vừa đắt hơn vừa kém ổn định số học.

Trên hàm mục tiêu (1.1), Newton với $t = 1$ hội tụ sau đúng một vòng lặp và cho $f - f^*$ bằng 0 đúng nghĩa trong `float64`. Kết quả này suy ra thẳng từ (1.2), vì Hessian là hằng số nên xấp xỉ bậc hai trùng khít hàm thật, và với $w_0 = 0$ thì

$$w_1 = w_0 - (\nabla^2 f)^{-1} \nabla f(w_0) = \left(\frac{1}{n} X^\top X + \lambda I \right)^{-1} \frac{1}{n} X^\top y = w^*, \quad (7.1)$$

tức bước Newton (7.1) chính là nghiệm đóng (1.3). Hình 7.1 vẽ cả bốn biến thể. Backtracking cũng vì thế không có việc gì để làm: điều kiện giảm đủ thỏa ngay ở lần thử thứ nhất, do được đúng một lần đánh giá hàm mục tiêu cho toàn bộ lần chạy. Phần backtracking cho Newton vì thế chỉ có nội dung khi hàm mục tiêu không còn bậc hai, chẳng hạn khi thay mất mát bình phương bằng mất mát Huber; chương 8 chạy lại đúng bốn biến thể này trên hàm mục tiêu đó.

7.2 Giá của một vòng lặp

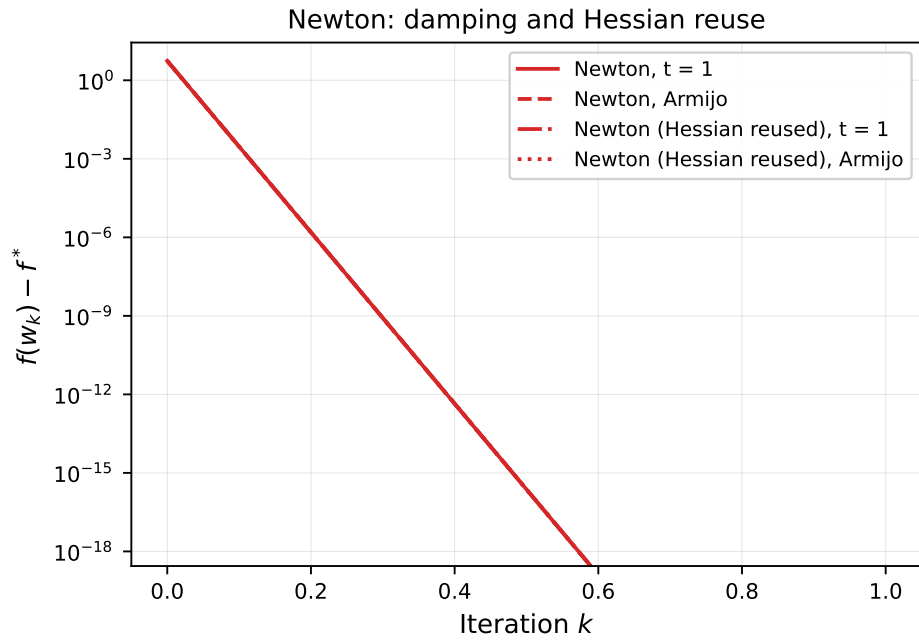
Điều đáng so sánh với Newton không phải số vòng lặp mà là chi phí của vòng lặp duy nhất ấy. Bảng 7.1 tách chi phí đó thành ba phần, đo ở quy mô $n = 1\,200\,000$.

Một bước Newton đầy đủ tốn khoảng 250 ms, trong đó phần $O(d^3)$ chỉ chiếm 0,2 ms, tức hoàn toàn không đáng kể. Với $d = 116$ và n cỡ triệu, chi phí dựng Hessian chỉ đắt hơn một lần tính gradient khoảng hai lần chứ không phải d lần như dự đoán thô ban đầu, vì phép nhân ma trận với ma trận thuộc nhóm BLAS bậc ba, đọc mỗi phần tử một lần rồi dùng lại trong bộ nhớ đệm, trong khi tính gradient phải quét toàn bộ ma trận hai lượt nên băng thông bộ nhớ chặn nó lại. Kết luận đảo chiều khi d lên tới vài nghìn, vì chi phí Hessian tăng theo d^2 còn chi phí gradient chỉ tăng theo d .

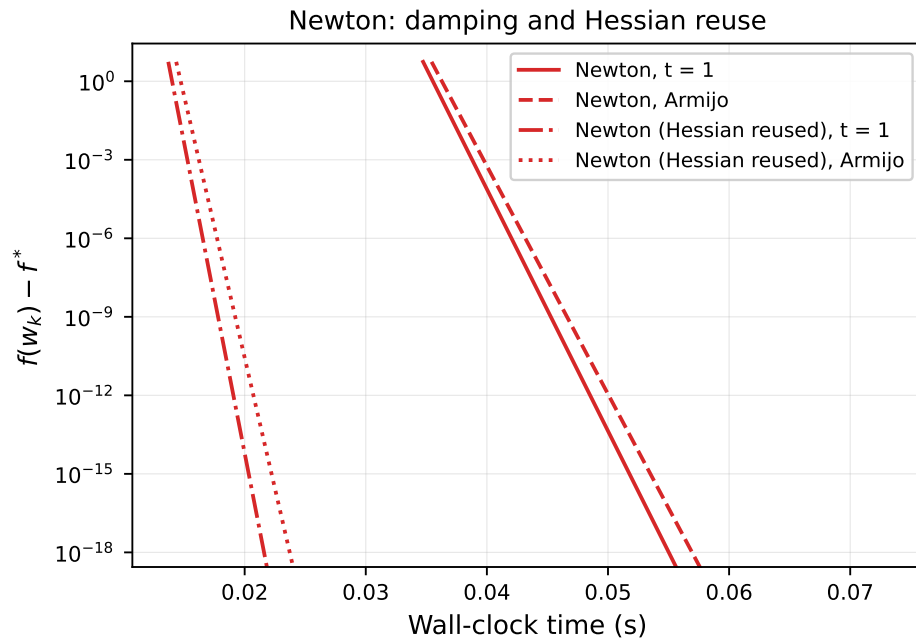
Bảng 7.1: Chi phí từng phần của một bước Newton, $n = 1\,200\,000$, $d = 116$.

Thành phần	Thời gian (ms)	Bậc chi phí
Dựng $\frac{1}{n} X^\top X$	163,2	$O(nd^2)$
Phân rã Cholesky 116×116	0,2	$O(d^3)$
Một lần tính gradient	87,3	$O(nd)$

(a) theo số vòng lặp



(b) theo thời gian chạy



Hình 7.1: Newton với bước đầy đủ và bước damped, hai cách xử lý Hessian.

Một đính chính về cách đo cần nêu rõ. Biến thể dùng lại phân rã Cholesky đo được 0,16 giây, nhưng con số đó bỏ sót phần dựng Hessian, vì thủ tục làm nóng chạy trước khi bấm đồng hồ đã tính sẵn ma trận đó. Biến thể dựng lại Hessian mỗi vòng đo được 0,40 giây, cao hơn 250 ms vì vòng lặp gọi thêm một lần đề xuất bước ở vòng thứ hai chỉ để kiểm tra điều kiện dừng, và với Newton thì lần gọi đó kéo theo cả việc dựng lại Hessian. Con số dùng để so sánh trong báo cáo là 250 ms.

Chương 8

Khi hàm mục tiêu không còn là hàm bậc hai

8.1 Mất mát Huber trên cùng bộ dữ liệu

Hàm mục tiêu thứ hai của báo cáo giữ nguyên ma trận thiết kế, vector mục tiêu và hệ số $\lambda = 0,031623$ của (1.1), chỉ thay mất mát bình phương bằng mất mát Huber [10]:

$$f_H(w) = \frac{1}{n} \sum_{i=1}^n H_\delta(x_i^\top w - y_i) + \frac{\lambda}{2} \|w\|^2, \quad H_\delta(r) = \begin{cases} \frac{1}{2}r^2, & |r| \leq \delta, \\ \delta \left(|r| - \frac{\delta}{2}\right), & |r| > \delta. \end{cases} \quad (8.1)$$

Phần dư nhỏ hơn ngưỡng δ vẫn bị phạt theo bình phương, phần dư lớn hơn chỉ bị phạt tuyến tính, nên một khoản vay ngoại lai không còn kéo được nghiệm về phía nó. Với báo cáo này thì tính chất thống kê đó là phân phụ; nhóm chọn (8.1) vì nó bỏ đúng một tính chất, tính bậc hai, mà giữ nguyên dữ liệu, số chiều và hệ số hiệu chỉnh, nên mọi chênh lệch đo được sau đây quy về đúng một nguyên nhân.

Ngưỡng đặt $\delta = 4,0666$, bằng 1,345 lần ước lượng bền vững của độ tán phần dư, tức trung vị độ lệch tuyệt đối chia cho 0,6745. Hằng số 1,345 là hằng số cổ điển, cho hiệu quả tiệm cận 95 phần trăm so với mất mát bình phương khi nhiều thật sự là nhiễu Gauss [10]. Độ tán ước lượng qua trung vị độ lệch tuyệt đối chứ không qua độ lệch chuẩn, vì chính cái đuôi nặng mà hàm mất mát này sinh ra để xử lý sẽ đẩy độ lệch chuẩn lên và kéo theo cả ngưỡng đáng lẽ dùng để cắt nó. Tại w^* , ngưỡng đó đặt 19,6 phần trăm số dòng ra ngoài vùng bậc hai, nên hàm mục tiêu khác Ridge trên số liệu chứ không chỉ khác trên công thức.

Hằng số Lipschitz của hai bài toán bằng nhau tới chữ số cuối, 9,1156, trong khi μ tụt từ 0,034073 xuống 0,031623 và κ vì thế tăng từ 267,5 lên 288,3 (bảng 8.1). Mỗi dòng dữ liệu đi vào Hessian của (8.1) với trọng số 1 hoặc 0 tùy phần dư của nó nằm trong hay ngoài vùng bậc hai, nên Hessian Huber không bao giờ vượt Hessian Ridge theo thứ tự nửa xác định dương và cận trên của Ridge dùng lại được nguyên vẹn.

Bảng 8.1: Ba hằng số của hai hàm mục tiêu trên cùng bộ dữ liệu, với $n = 200\,000$, $d = 116$ và $\lambda = 0,031623$ cho cả hai. Cặp cột μ và κ thứ nhất là cận bảo đảm, thứ mà thuật toán dùng được; cặp thứ hai đo tại w^* . Với Ridge hai cặp trùng nhau vì Hessian là hằng số.

Hàm mục tiêu	δ	L	Cận bảo đảm		Đo tại w^*		f^*
			μ	κ	μ	κ	
Ridge	–	9,1156	0,034073	267,5	0,034073	267,5	6,151114
Huber	4,0666	9,1156	0,031623	288,3	0,033609	271,2	5,196306

Cận dưới thì không dùng lại được. Trường hợp xấu nhất là mọi dòng đều nằm ngoài vùng bậc hai, khi đó phần mất mát không góp chút độ cong nào và chỉ còn số hạng hiệu chỉnh, nên μ chỉ bảo đảm được ở mức λ .

Do tại w^* , trị riêng nhỏ nhất của Hessian là 0,033609, lớn hơn cận bảo đảm 6,3 phần trăm, và số điều kiện thật là 271,2 chứ không phải 288,3. Khoảng cách giữa hai con số có nguồn rõ ràng, vì tại w^* chỉ 19,6 phần trăm số dòng rơi ra ngoài vùng bậc hai còn hơn bốn phần năm số dòng vẫn góp độ cong đúng như ở Ridge. Báo cáo ghi cả hai giá trị vì chúng phục vụ hai việc khác nhau. Thuật toán chỉ dùng được λ , do gradient tăng tốc đọc μ để dựng hệ số quán tính mà μ tại w^* thì chỉ biết sau khi đã có w^* . Tốc độ quan sát được, ngược lại, bám theo 271,2, nên mọi so sánh với cận lý thuyết tính từ $\kappa = 288,3$ sẽ thấy thực nghiệm nhanh hơn cận. Nhận xét đó chỉ đúng quanh w^* ; nếu dấy lập đi qua một vùng có nhiều dòng nằm ngoài hơn thì μ cục bộ ở đó còn thấp hơn giá trị đo tại w^* .

Hàm (8.1) không có nghiệm đóng, nên $f^* = 5,196306$ phải giải bằng số: một lần chạy Newton riêng, độc lập với mọi lần chạy được đo, dừng sau 8 vòng lặp với $\|\nabla f_H(w^*)\| = 2,6 \cdot 10^{-16}$. Mất mát lớn hơn nằm ở cách đo sai số. Đẳng thức (1.5) dựa vào việc khai triển Taylor của hàm bậc hai không còn số hạng dư, mà (8.1) không có đẳng thức nào tương ứng, nên gap chỉ còn tính được bằng phép trừ, với phép trừ dồn xuống từng dòng thay vì làm một lần ở cuối. Đo trên chính bài toán này, cách dồn xuống từng dòng hạ sàn từ $8,9 \cdot 10^{-16}$ xuống $1,1 \cdot 10^{-16}$, tức khoảng 7,75 lần, và đổi cửa sổ lấy trung vị thì tỷ lệ nằm trong khoảng 7,75 tới 8,5 lần. Mức cải thiện đó chưa tới một bậc độ lớn, trong khi (1.5) giành thêm hơn tám bậc trên bài toán thử $n = 400, d = 12$ của mục 1.3. Hai con số đo trên hai bài toán khác nhau nên không so trực tiếp được, nhưng khoảng cách giữa chúng đủ lớn để giải thích vì sao mọi hình trong chương này có sàn cao hơn hẳn các hình trước đó.

Cách dồn theo dòng hạ được sàn nhưng không giữ được dấu. Đo dọc một hướng ngẫu nhiên cố định, nó bắt đầu tách khỏi giá trị thật ở khoảng cách 10^{-7} tới w^* mà vẫn còn dương, rồi đổi dấu ngay ở khoảng cách kế tiếp: $-1,3 \cdot 10^{-16}$ ở 10^{-8} , và $-1,2 \cdot 10^{-17}$ ở 10^{-13} . Một giá trị âm không có chỗ trên trục logarit, nên điểm đó không mờ dần xuống đáy mà biến mất khỏi trục và bị kẹp về đáy khung hình. Các hình của chương này giữ nguyên phần đuôi thay vì cắt bớt, nên dải răng cưa dưới khoảng 10^{-15} phải đọc là sàn của phép đo, không phải dao động của thuật toán.

8.2 Newton mất đặc quyền một vòng

Newton cần 7 vòng lặp để đạt ngưỡng 10^{-6} và 8 vòng để chạm sàn số, thay vì đúng một vòng như trên Ridge. Bốn vòng đầu chỉ đưa sai số từ 3,85 xuống 0,19, còn từ vòng thứ năm trở đi thì số mũ gần như nhân đôi mỗi vòng, $1,10 \cdot 10^{-2}$ rồi $1,98 \cdot 10^{-5}$, $6,65 \cdot 10^{-11}$ và $1,66 \cdot 10^{-17}$, đúng dạng hội tụ bậc hai [9] (cột thứ hai của bảng 8.2). Hessian của (8.1) đổi theo w vì tập dòng nằm trong vùng bậc hai đổi theo w , nên xa w^* thì xấp xỉ bậc hai còn số hạng dư đáng kể và bước Newton chỉ là một bước giảm bình thường; gần w^* thì tập đó ổn định lại, số hạng dư teo theo bình phương sai số, và tốc độ bậc hai xuất hiện.

Dùng lại phân rã Cholesky của điểm khởi tạo, thứ đã cho Newton kết quả tốt nhất trên Ridge, thì với bước đầy đủ dấy lập không hội tụ. Sai số khóa vào một chu trình hai điểm, luân phiên giữa 2,3064 và 2,6076, và chạy hết ngân sách 60 vòng vẫn không thoát ra; hình 8.1 vẽ nó thành một đường nằm ngang. Ma trận đóng băng tại $w_0 = 0$

Bảng 8.2: Sai số $f_H(w_k) - f^*$ từng vòng của ba biến thể Newton, cùng điểm khởi tạo $w_0 = 0$ và cùng ngân sách 60 vòng. Cột Newton dừng ở vòng 8 vì đã chạm sàn số. Hai cột sau dùng lại phân rã Cholesky của w_0 , tức phương pháp dây cung.

Vòng	Newton, $t = 1$	Dây cung, $t = 1$	Dây cung, Armijo
0	3,8475	3,8475	3,8475
1	2,8374	2,8374	2,8374
2	1,1954	2,3728	2,3728
3	0,6174	2,6264	$1,66 \cdot 10^{-2}$
4	0,1928	2,3135	$1,60 \cdot 10^{-3}$
5	$1,10 \cdot 10^{-2}$	2,6086	$1,60 \cdot 10^{-3}$
6	$1,98 \cdot 10^{-5}$	2,3070	$2,12 \cdot 10^{-4}$
7	$6,65 \cdot 10^{-11}$	2,6073	$7,67 \cdot 10^{-5}$
8	$1,66 \cdot 10^{-17}$	2,3064	$3,83 \cdot 10^{-5}$
13		2,6076	$5,32 \cdot 10^{-7}$
60		2,3064	$1,15 \cdot 10^{-16}$

mang độ cong tại điểm xuất phát chứ không phải tại điểm hiện tại, nên bước đầy đủ theo hướng đó vượt qua cực tiểu và rơi sang phía đối diện với sai số gần bằng sai số cũ, rồi vòng sau lại vượt ngược về. Tên đúng của biến thể này là phương pháp dây cung, và trên Ridge thì hai tên gọi trùng nhau, vì Hessian là hằng số nên ma trận đóng băng bằng đúng ma trận thật và bảng 9.1 ghi nó hội tụ trong một vòng.

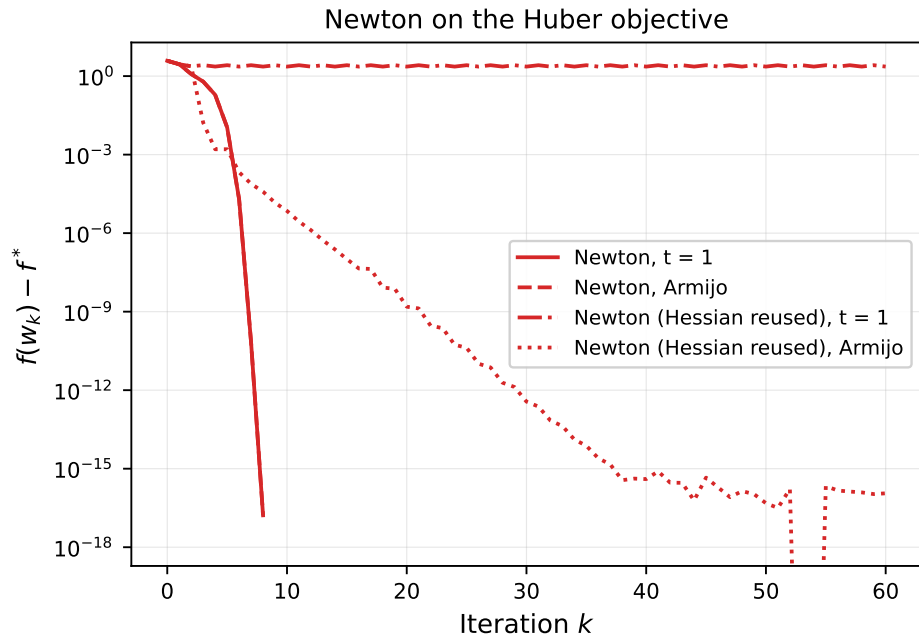
Thêm backtracking vào đúng hướng đó thì nó hội tụ, đạt 10^{-6} ở vòng 13 và xuống $1,15 \cdot 10^{-16}$ ở cuối ngân sách. Hướng dây cung luôn là hướng giảm, vì ma trận đóng băng xác định dương nên $\nabla f_H^\top p = -\nabla f_H^\top H(w_0)^{-1} \nabla f_H < 0$, và độ dài bước mới là chỗ hổng; line search cắt bước xuống dưới ngưỡng vượt và trả lại tốc độ tuyến tính mà phương pháp dây cung có damping phải có. Từ vòng 6 trở đi, cột cuối của bảng 8.2 cho thấy sai số giảm còn khoảng 0,4 tới 0,5 lần mỗi vòng, một hệ số gần như cố định, khác hẳn cột Newton bên cạnh. Trên hàm mục tiêu này, line search không phải một cải tiến của phương pháp dây cung mà là điều kiện để nó chạy được.

Dựng lại Hessian mỗi vòng, thứ mà biến thể đóng băng muốn tránh, tốn 26,3 ms ở quy mô $n = 200\,000$, so với 20,8 ms của Hessian Ridge trên cùng dữ liệu. Chênh lệch chỉ khoảng một phần tư vì chỉ các dòng nằm trong vùng bậc hai đi vào tích $X_S^\top X_S$, và số dòng đó nhỏ hơn n ; cách cài đặt còn lại, giữ đủ n dòng rồi nhân trọng số 1 hoặc 0, đo được 64,1 ms, chậm hơn 2,44 lần. Với giá đó thì đóng băng Hessian không đáng, vì nó tiết kiệm 7 lần dựng ma trận, mỗi lần 26,3 ms, để đổi lấy 6 vòng lặp thừa và một line search bắt buộc. Cân nhắc này đảo chiều khi d lớn, do chi phí dựng Hessian tăng theo d^2 còn phần tiết kiệm chỉ tăng tuyến tính theo số vòng.

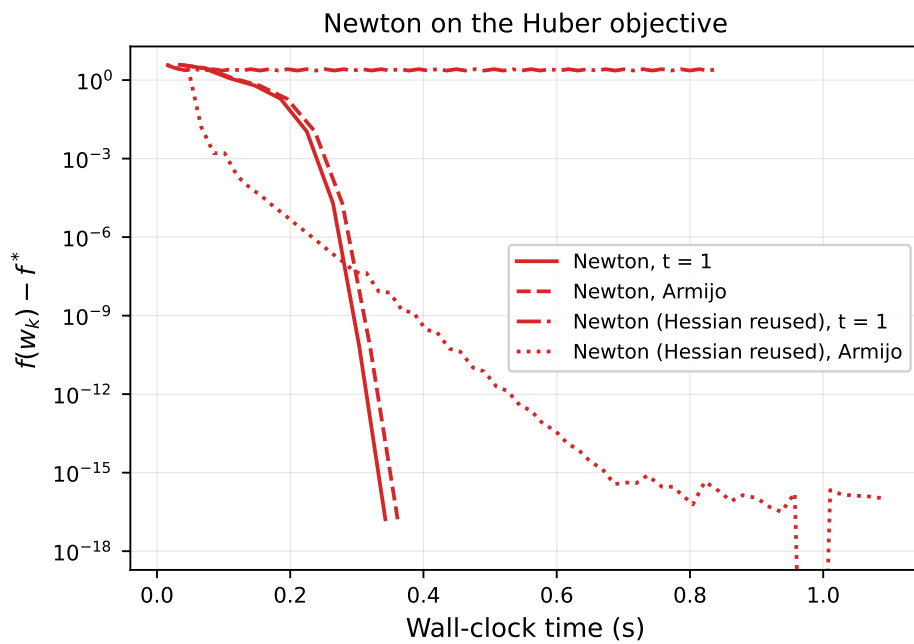
8.3 Backtracking có việc với hướng nào

Với hướng gradient, backtracking lùi bước thật, tốn trung bình 4,20 lần đánh giá hàm mục tiêu mỗi vòng lặp. Con số đó gần như không đổi so với Ridge, nơi cùng cấu hình $c = 0,3$, $\rho = 0,5$, $t_0 = 1$ tốn 4,94 lần (bảng 4.2), nên phần này của kết quả không nói gì về hình dạng hàm mục tiêu. Line search đã có việc từ trước, vì hướng gradient không

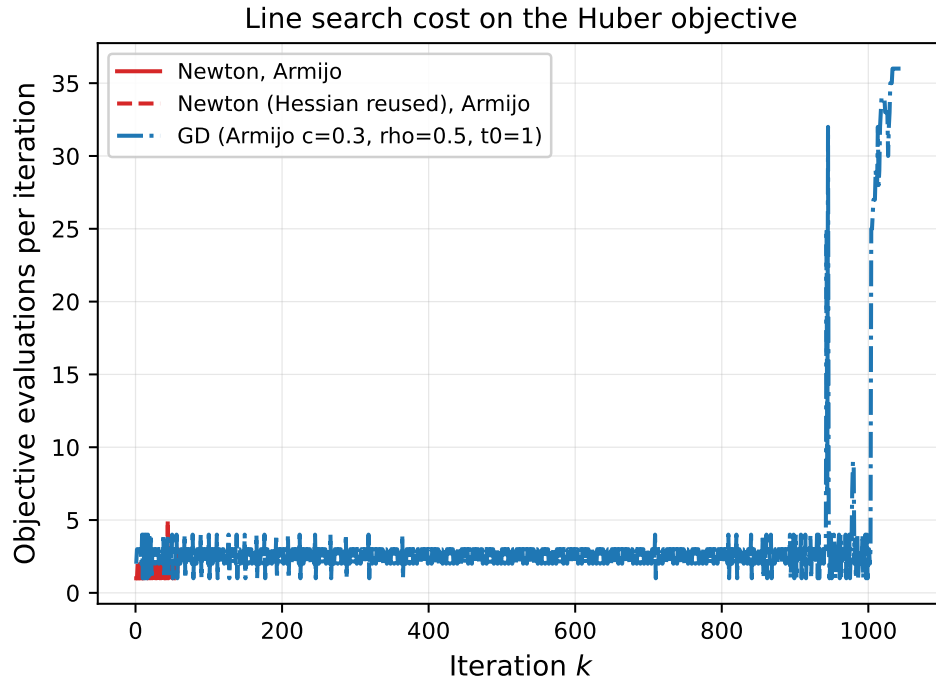
(a) theo số vòng lặp



(b) theo thời gian chạy



Hình 8.1: Newton trên hàm mục tiêu Huber, với bước đầy đủ và với backtracking, mỗi biến thể một lần dựng lại Hessian và một lần dùng lại nhân tử của điểm khởi tạo. Hình chỉ thấy ba đường vì hai biến thể dựng lại Hessian mỗi vòng, một với bước đầy đủ và một với backtracking, cho cùng một dãy sai số nên vẽ chồng khít lên nhau; backtracking nhận bước đầy đủ ở mọi vòng nên không có gì tách chúng ra.



Hình 8.2: Số lần đánh giá hàm mục tiêu mỗi vòng lặp mà điều kiện Armijo đòi, trên hướng gradient so với trên hướng Newton. Ba đường lấy từ hai nhóm thí nghiệm nên tham số c không giống nhau: $c = 0,3$ cho gradient descent, $c = 10^{-4}$ cho hai đường Newton. Phép so sánh không phụ thuộc c là phép so giữa hai đường Newton với nhau.

mang thông tin độ cong ở cả hai bài toán và $t_0 = 1$ vì thế luôn nằm xa trên bước mà điều kiện Armijo chấp nhận.

Trung bình 4,20 không mô tả đúng phần thân lần chạy. Suốt 942 vòng đầu, chi phí dao động giữa 1 và 4 lần mỗi vòng với trung vị đúng bằng 3 và không vòng nào vượt 4; hai vòng 943 và 945 vọt lên 25 và 32 rồi rơi lại ngay, và từ vòng 946 tới vòng 1003 chi phí về đúng dải cũ, chỉ ba vòng lên tới 7 hoặc 9. Bức tường thật bắt đầu ở vòng 1004: 48 vòng cuối không vòng nào xuống dưới 25 và đỉnh là 36 lần ở vòng 1033 (hình 8.2). Ở phần đuôi đó, mức giảm thật của hàm mục tiêu đã nhỏ hơn nhiều làm tròn của f_H nên mọi bước thử đều trượt điều kiện giảm đủ và line search phải chia đôi hàng chục lần mới tìm được một bước lọt qua, đúng hiện tượng mà chương 4 mô tả bằng hình 4.2. Bức tường đó cũng là lý do lần chạy kết thúc ở trạng thái đình trệ chứ không phải hội tụ.

Với hướng Newton, backtracking không lùi bước một lần nào, đo được đúng 1,00 lần đánh giá hàm mỗi vòng ở mọi vòng của lần chạy, tức line search nhận ngay bước đầy đủ ở lần thử thứ nhất. Dự đoán ban đầu của nhóm là ngược lại, rằng bỏ tính bậc hai sẽ làm backtracking có việc với cả hai hướng, và dự đoán đó sai vì nó lẫn hai chuyện. Hàm mục tiêu hết bậc hai thì bước Newton hết trúng nghiệm, nhưng nó vẫn là bước cực tiểu hóa xấp xỉ bậc hai tại chỗ, nên mức giảm mà nó hứa đã tính theo độ cong và điều kiện giảm đủ thỏa ngay.

Phép đối chứng sạch cho khẳng định trên nằm ở cặp Newton và dây cung, vì hai biến thể đó chia nhau đúng một cấu hình line search với $c = 10^{-4}$: hướng Newton tốn 1,00 lần mỗi vòng còn hướng dây cung tốn 1,53 lần, tức bị từ chối ở khoảng một

Bảng 8.3: Bốn thuật toán trên hàm mục tiêu Huber, mỗi cấu hình giữ nguyên như đã chọn cho bài toán Ridge, đặt cạnh cột Ridge lấy từ bảng 9.1. Hai cột đầu đo tới ngưỡng 10^{-6} , các cột sau là toàn bộ lần chạy. Dòng Newton của cột Ridge là biến thể dùng lại Hessian, vốn trùng với Newton trên hàm bậc hai; trên Huber biến thể đó là phương pháp dây cung nên chỗ này dùng Newton dựng lại Hessian mỗi vòng. SGD không đạt 10^{-6} và dừng ở $3,1 \cdot 10^{-4}$.

Cấu hình	Huber				Ridge	
	Tới 10^{-6}		Chạy hết		Chạy hết	
	Vòng	Giây	Vòng	Giây	Vòng	Giây
Newton, $t = 1$	7	0,34	8	0,38	1	0,03
AGD, β theo t và μ	83	1,19	301	4,29	286	3,92
GD, backtracking	220	4,83	1051	27,21	561	13,86
GD, $t = 1,9/L$	381	5,44	2108	30,64	2022	27,52
SGD, $B = 2048$	–	–	3880	2,27	3880	2,02

nửa số vòng, do ma trận đóng băng chỉ mang độ cong tại điểm xuất phát. Phép so 4,20 với 1,00 thì không sạch bằng, vì nhóm gradient đo với $c = 0,3$ tức một điều kiện giảm đủ chặt hơn ba nghìn lần, nên nó có phần lợi cho Newton và chỉ nên đọc như một mốc độ lớn.

Đổi số vòng lấy giá mỗi vòng vẫn có lời trên hàm mục tiêu này, nhưng phần lời thu hẹp. Gradient descent với backtracking đạt 10^{-6} ở vòng 220 so với 381 vòng của bước cố định $1,9/L$, và tính theo thời gian thì 4,83 giây so với 5,44 giây, tức nhanh hơn 13 phần trăm. Trên Ridge, chương 4 đo được 143 vòng và 3,14 giây so với 331 vòng và 4,75 giây, tức nhanh hơn một nửa. Mỗi lần đánh giá hàm phải nhân Xw , cùng bậc $O(nd)$ với một lần tính gradient, nên ba lần thử mỗi vòng xóa gần hết lợi thế 1,7 lần về số vòng lặp.

8.4 Thứ hạng bốn thuật toán trên hàm phi tuyến

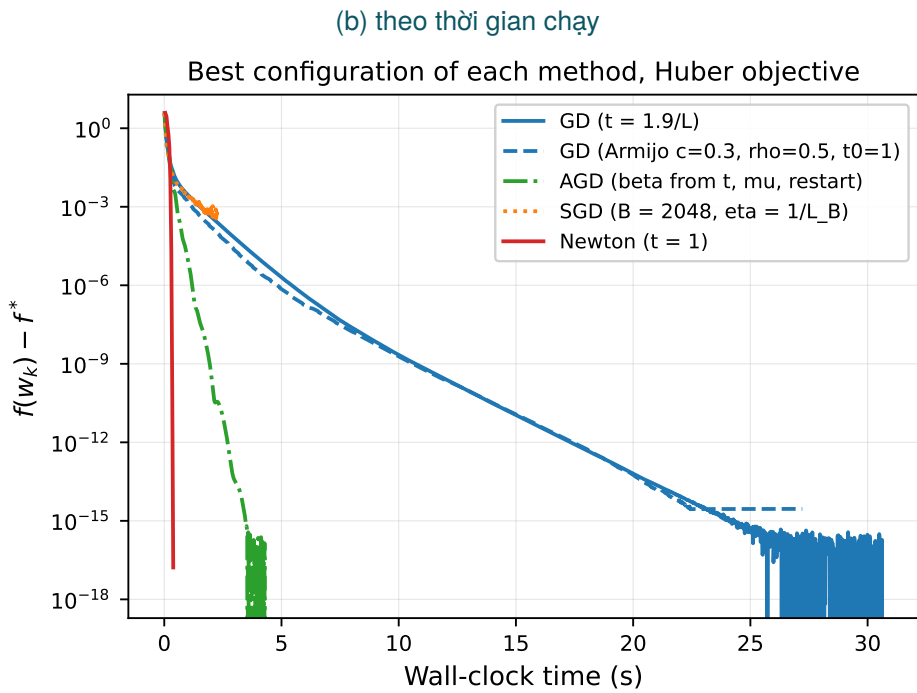
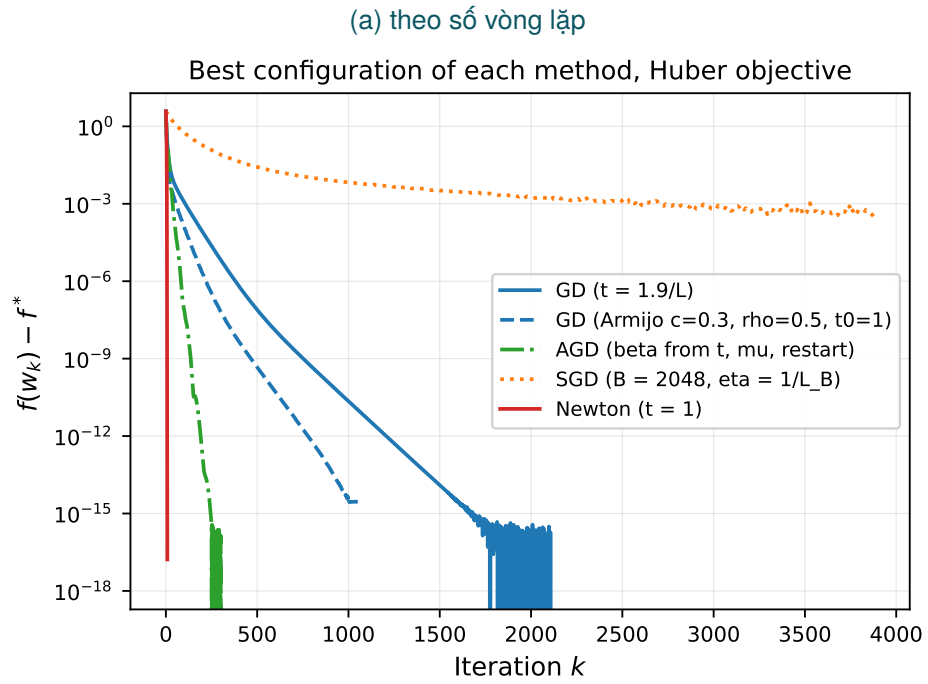
Thứ hạng bốn thuật toán trên hàm mục tiêu Huber giống thứ hạng trên Ridge, và giống nhau trên cả hai trục đo: Newton 7 vòng và 0,34 giây, gradient tăng tốc 83 vòng và 1,19 giây, gradient descent với backtracking 220 vòng và 4,83 giây, rồi bước cố định 381 vòng và 5,44 giây (bảng 8.3). Giá mỗi vòng của bốn phương pháp chênh nhau ít hơn nhiều so với số vòng, nên trục thời gian không đủ sức đảo trật tự mà trục vòng lặp đã đặt ra. Cơ chế này giống hết cơ chế mà chương 9 nêu cho bài toán Ridge, và nó đảo chiều ở cùng một điều kiện, khi d lớn tới mức chi phí $O(nd^2)$ của Hessian chi phối.

Ba phương pháp bậc một gần như không nhận ra hàm mục tiêu đã đổi. Gradient descent với bước cố định chạy 2108 vòng trong 30,6 giây trên Huber so với 2022 vòng và 27,5 giây trên Ridge, còn gradient tăng tốc chạy 301 vòng và 4,29 giây so với 286 vòng và 3,92 giây. Số vòng lặp của phương pháp bậc một chỉ phụ thuộc κ , mà κ thật chỉ đi từ 267,5 lên 271,2, tức 1,4 phần trăm. Newton là phương pháp duy nhất thiệt hại thật sự, vì tính bậc hai chính là thứ đã cho nó một vòng lặp, và khoảng cách về số vòng giữa nó với gradient descent thu hẹp từ 2022 lần xuống 264 lần. Hai dòng

backtracking không đọc theo cách này được, do cả hai dừng ở trạng thái đình trệ nên 1051 vòng và 561 vòng đo vị trí sàn số của từng bài toán chứ không đo độ khó của chúng.

SGD chạy 3880 vòng lô trong 2,27 giây, nhanh nhất trên mỗi đơn vị thời gian, nhưng dừng ở sai số $3,1 \cdot 10^{-4}$ và không bao giờ tới 10^{-6} , sát mức $4,9 \cdot 10^{-4}$ mà cùng cấu hình $B = 2048$ đặt trên Ridge ở cùng quy mô $n = 200\,000$. Sàn đó do phương sai gradient lô quyết định chứ không do độ cong của hàm mục tiêu, nên đổi mất mát bình phương sang mất mát Huber không dịch nó đi đáng kể. Ở ngưỡng 10^{-3} thì SGD vẫn thắng cả bốn phương pháp còn lại; ở ngưỡng chặt hơn thì nó không bao giờ về đích.

Phần đuôi của hình 8.3 phải đọc theo sàn số đã nêu ở mục 8.1. Dưới khoảng 10^{-15} , đường gradient descent và đường gradient tăng tốc chuyển thành một dải răng cưa biên độ cỡ 10^{-16} , còn đường dây cung có damping trong hình 8.1 rơi thẳng xuống đáy khung ở vòng 53 và vòng 54, hai vòng duy nhất của cả lần chạy mà công thức trả về giá trị âm. Ở cả hai chỗ đó, sai số đo được đã xuống dưới sàn của phép trừ, và một giá trị âm thì bị kẹp lên đáy trục logarit. Đường Newton dừng ở $1,66 \cdot 10^{-17}$ sau 8 vòng cũng vì chạm sàn đó chứ không vì thuật toán hết khả năng giảm tiếp. Các hình giữ nguyên phần đuôi thay vì cắt bớt, nên người đọc thấy đúng chỗ mà phép đo hết tác dụng.



Hình 8.3: Bốn thuật toán trên hàm mục tiêu Huber, mỗi thuật toán giữ nguyên cấu hình đã chọn cho bài toán Ridge ở chương 9.

Chương 9

So sánh tổng hợp trên hai trục

9.1 Cấu hình tốt nhất của từng phương pháp

Bảng 9.1 tập hợp cấu hình tốt nhất tìm được ở các chương trước, đo lại ở cả hai quy mô mẫu với ba lần chạy độc lập mỗi cấu hình và lấy trung vị.

Thứ hạng theo hai trục đo giống nhau ở bài toán này, khác với tình huống mà đề bài dự đoán. Newton đứng đầu cả về số vòng lặp lẫn thời gian, và khoảng cách theo thời gian với gradient descent là gần một nghìn lần ở quy mô toàn phần. Lý do là giá mỗi vòng của bốn phương pháp chênh nhau ít hơn số vòng lặp chênh nhau, vì một vòng Newton đắt hơn một vòng gradient descent khoảng ba lần, trong khi gradient descent cần nhiều hơn hai nghìn lần số vòng. Thứ hạng sẽ đảo khi d lớn, vì lúc đó chi phí Hessian tăng theo d^2 .

SGD là ngoại lệ và phải đọc riêng. Nó nhanh nhất trên mỗi đơn vị thời gian, đạt sai số $6,0 \cdot 10^{-4}$ chỉ sau 13,17 giây ở quy mô toàn phần, nhưng không bao giờ tới ngưỡng 10^{-6} vì sần nhiễu mô tả ở chương 6. Nếu bài toán chỉ đòi độ chính xác cỡ 10^{-3} thì SGD thắng tuyệt đối; ở ngưỡng chặt hơn thì nó không phải một lựa chọn. Hình 9.1 và hình 9.2 vẽ hai quy mô.

9.2 Ảnh hưởng của kích thước mẫu

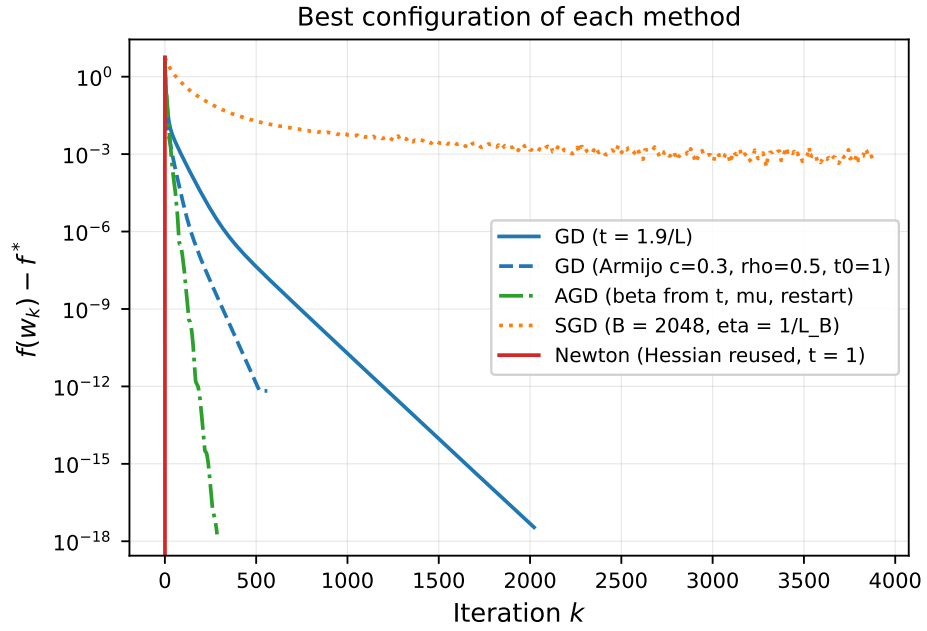
So sánh hai nửa của bảng 9.1 cho thấy số vòng lặp gần như không đổi khi n tăng sáu lần: gradient descent đi từ 2022 xuống 1959 vòng, gradient tăng tốc từ 286 xuống 283. Thời gian thì tăng từ 5,2 tới 6,5 lần, tức tuyến tính theo n đúng như mô hình chi phí $O(nd)$ mỗi vòng.

Hai quan sát này là hệ quả trực tiếp của hệ số $\frac{1}{n}$ trong (1.1). Nó giữ κ gần như cố định giữa hai quy mô, 267,5 so với 266,2, mà số vòng lặp của mọi phương pháp bậc một lại chỉ phụ thuộc κ chứ không phụ thuộc n . Bỏ hệ số đó đi thì L tăng tuyến tính

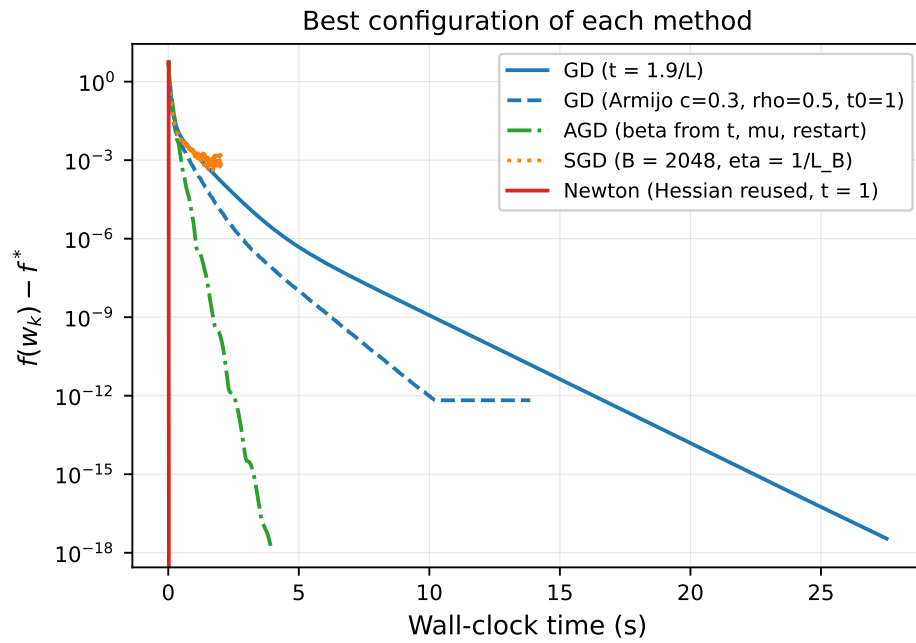
Bảng 9.1: Cấu hình tốt nhất của từng phương pháp ở hai quy mô. Cột số vòng lặp là số vòng chạy hết, cột thời gian là thời gian tương ứng.

Cấu hình	$n = 200\,000$		$n = 1\,200\,000$	
	Vòng lặp	Giây	Vòng lặp	Giây
Newton, Hessian dùng lại	1	0,03	1	0,16
AGD, β theo t và μ	286	3,92	283	23,49
GD, backtracking	561	13,86	514	71,99
GD, $t = 1,9/L$	2022	27,52	1959	158,67
SGD, $B = 2048$	3880	2,02	23400	13,17

(a) theo số vòng lặp

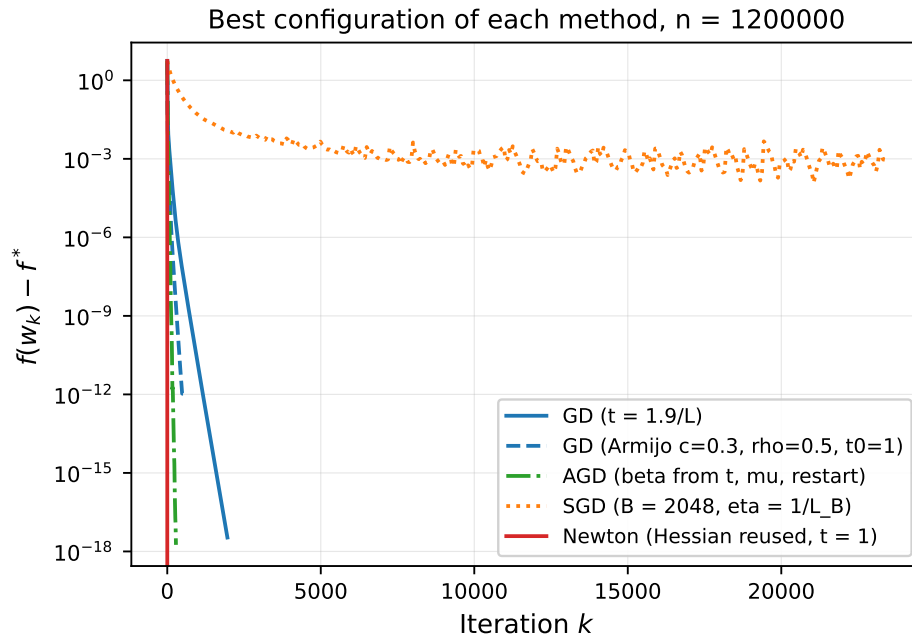


(b) theo thời gian chạy

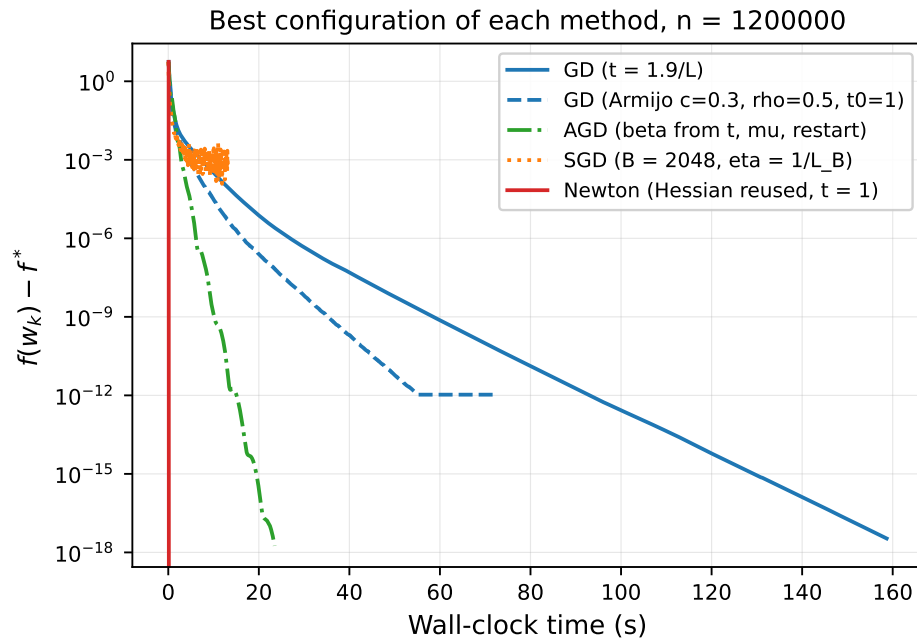


Hình 9.1: Cấu hình tốt nhất của từng phương pháp, $n = 200\,000$.

(a) theo số vòng lặp



(b) theo thời gian chạy



Hình 9.2: Cấu hình tốt nhất của từng phương pháp, $n = 1\,200\,000$.

Bảng 9.2: Tỷ lệ số vòng lặp giữa các cặp κ liên kề, so với tỷ lệ mà lý thuyết dự đoán.

κ giảm	Tỷ lệ κ	GD quan sát	AGD quan sát	$\sqrt{\kappa}$ dự đoán
730 xuống 268	2,73	2,73	1,57	1,65
268 xuống 90	2,98	2,98	1,56	1,73
90 xuống 10	8,91	8,86	2,82	2,99

theo n và toàn bộ lưới tham số phải quét lại ở mỗi quy mô. SGD là ngoại lệ vì ngân sách của nó đặt theo số epoch, mà một epoch ở quy mô lớn chứa nhiều lô hơn.

9.3 Đối chiếu với tốc độ lý thuyết

Lý thuyết cho hàm bậc hai lồi mạnh nói số vòng lặp của gradient descent tỷ lệ với κ , còn của gradient tăng tốc tỷ lệ với $\sqrt{\kappa}$ [5]. Bảng 9.2 đối chiếu bằng cách đo trên cùng bộ dữ liệu với các giá trị λ khác nhau, mỗi lần dùng bước tính theo L và μ của chính bài toán đó.

Cột gradient descent khớp cận lý thuyết tới hai chữ số ở cả ba cặp. Cột gradient tăng tốc bám sát $\sqrt{\kappa}$ nhưng lệch chừng năm tới mười phần trăm, và lệch theo hướng tốt hơn dự đoán ở cặp cuối. Hằng số nhân trước lũy thừa trong cận tăng tốc phụ thuộc κ , nên tỷ lệ giữa hai lần chạy không rút gọn sạch như ở cận của gradient descent.

Chương 10

So sánh với scikit-learn

10.1 Quy đổi hàm mục tiêu

Điểm dễ sai nhất khi so sánh với thư viện là mỗi ước lượng chuẩn hóa hàm mục tiêu theo một cách khác nhau. Lớp `Ridge` của scikit-learn [11] cực tiểu hóa $\|Xw - y\|^2 + \alpha\|w\|^2$, tức bỏ cả hệ số $\frac{1}{n}$ lẫn hệ số $\frac{1}{2}$ của (1.1); nhân (1.1) với $2n$ cho quan hệ

$$\alpha_{\text{Ridge}} = \lambda n. \quad (10.1)$$

Lớp `SGDRegressor` lại cực tiểu hóa $\frac{1}{n} \sum_i \frac{1}{2} (x_i^\top w - y_i)^2 + \frac{\alpha}{2} \|w\|^2$, tức trùng đúng với (1.1), nên với nó $\alpha = \lambda$. Hai ước lượng trong cùng một thư viện cần hai hằng số khác nhau, và đặt sai không sinh ra lỗi nào: cả hai bên vẫn giải sạch, chỉ là giải hai bài toán khác nhau.

Quy đổi (10.1) được kiểm chứng bằng số chứ không bằng suy luận trên giấy. Lấy nghiệm $\hat{w}$ do `Ridge` trả về với $\alpha = \lambda n = 3,795 \cdot 10^4$ rồi so với nghiệm đóng cho $\|\hat{w} - w^*\|/\|w^*\| = 2,35 \cdot 10^{-15}$, tức ở mức sai số máy. Phép kiểm này chạy tự động trước mọi so sánh và làm dừng chương trình nếu không đạt.

10.2 Kết quả

Bảng 10.1 ghi kết quả ở quy mô toàn phần.

Mã tự viết và thư viện làm cùng một việc và mất cùng một thời gian: 0,25 giây so với 0,273 giây, với sai số của cả hai ở mức sai số máy. Hai con số sát nhau không nói bên nào nhanh hơn, mà là bằng chứng cả hai cài đúng, vì `Ridge` với solver mặc định cũng chính là nghiệm đóng giải qua Cholesky. Mục đích của việc tự cài đặt không phải chạy nhanh hơn thư viện, mà là hiểu cơ chế hội tụ và biết cách chọn tham số.

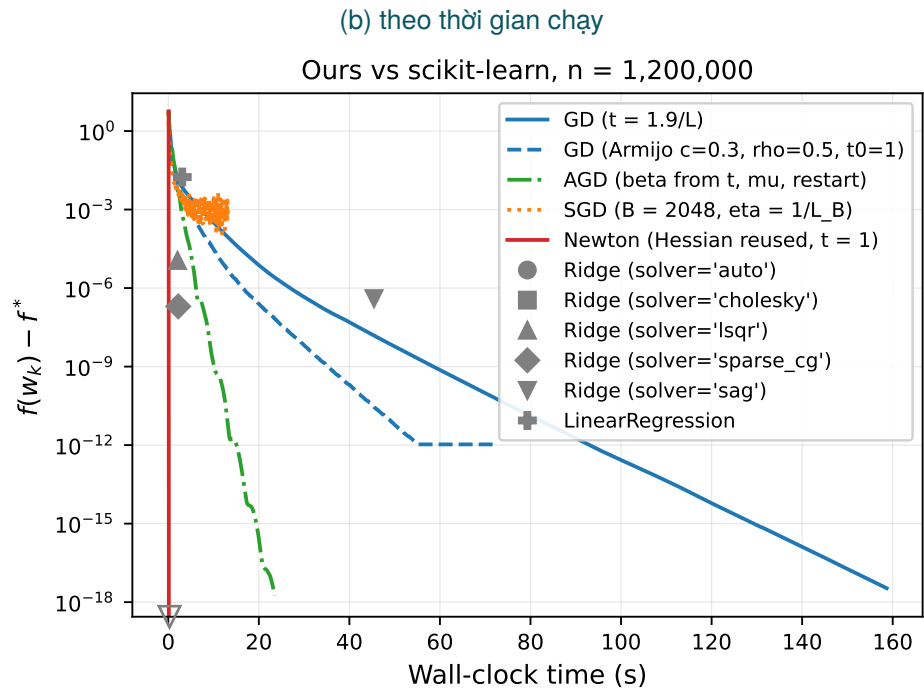
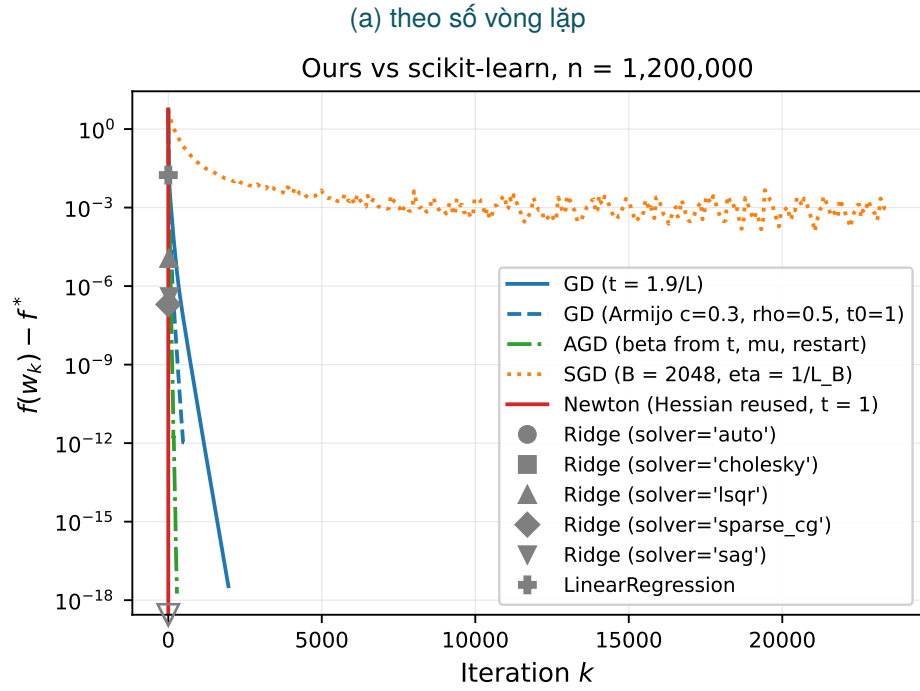
Bảng 10.1: So sánh với scikit-learn ở quy mô $n = 1\,200\,000$, trung vị của ba lần chạy. Dòng đầu là nghiệm đóng của nhóm.

Phương pháp	Thời gian (s)	$f - f^*$	RMSE test
Newton của nhóm	0,25	0	3,4646
<code>Ridge(cholesky)</code>	0,273	$4,15 \cdot 10^{-30}$	3,4646
<code>Ridge(lsqr)</code>	2,03	$1,13 \cdot 10^{-5}$	3,4647
<code>Ridge(sparse_cg)</code>	2,20	$1,99 \cdot 10^{-7}$	3,4646
<code>Ridge(sag)</code>	45,44	$3,96 \cdot 10^{-7}$	3,4647
<code>SGDRegressor</code>	3,18	$6,21 \cdot 10^{20}$	$3,3 \cdot 10^{10}$
<code>LinearRegression</code>	3,08	$1,75 \cdot 10^{-2}$	3,4614

Ba solver lặp của Ridge đổi độ chính xác lấy thời gian theo những tỷ lệ rất khác nhau. Solver `lsqr` dừng ở sai số $1,13 \cdot 10^{-5}$ sau 2,03 giây, còn `sag` tốn 45,44 giây để đạt $3,96 \cdot 10^{-7}$, tức chậm hơn nghiệm trực tiếp 166 lần mà vẫn kém chính xác hơn 23 bậc. Chúng chỉ có lợi khi d lớn tới mức chi phí $O(d^3)$ hoặc bộ nhớ cho $X^T X$ không chấp nhận được, điều không xảy ra với $d = 116$.

`SGDRegressor` với tham số mặc định cho sai số $6,21 \cdot 10^{20}$ và RMSE $3,3 \cdot 10^{10}$, tức hoàn toàn không dùng được, sau khi dừng ở vòng lặp thứ sáu theo tiêu chí dừng mặc định. Kết quả này là phát biểu về giá trị mặc định chứ không phải về thuật toán: SGD tự cài với bước đặt theo $1/L_B$ đạt sai số $6,0 \cdot 10^{-4}$ trên cùng bài toán. Chỉ cần thay tham số cho `SGDRegressor` sẽ cải thiện, nhưng mặc định của thư viện không đặt bước theo L của bài toán cụ thể.

Dòng `LinearRegression` ứng với $\lambda = 0$ nên giải một bài toán khác, và sai số $1,75 \cdot 10^{-2}$ so với f^* phản ánh đúng điều đó. Đáng chú ý là RMSE trên tập kiểm tra của nó, 3,4614, lại nhỏ hơn nghiệm Ridge một chút, 3,4646. Chương 11 bàn về khoản chênh này. Hình 10.1 đặt các kết quả thư viện cạnh đường hội tụ của mã tự viết.



Hình 10.1: So sánh mã tự viết với scikit-learn ở quy mô $n = 1\,200\,000$. Các kết quả thư viện là điểm đơn vì solver không cho lịch sử theo vòng lặp.

Chương 11

Ảnh hưởng của hệ số hiệu chỉnh

Hệ số λ có hai vai trò tách biệt. Vai trò thống kê là chống quá khớp, và vai trò tối ưu hóa là đặt sần cho phổ của Hessian: theo (1.4) thì $\mu = \lambda_{\min} + \lambda$, nên tăng λ làm tăng μ , giảm κ và do đó tăng tốc độ hội tụ. Bảng 11.1 đo cả hai vai trò cùng lúc.

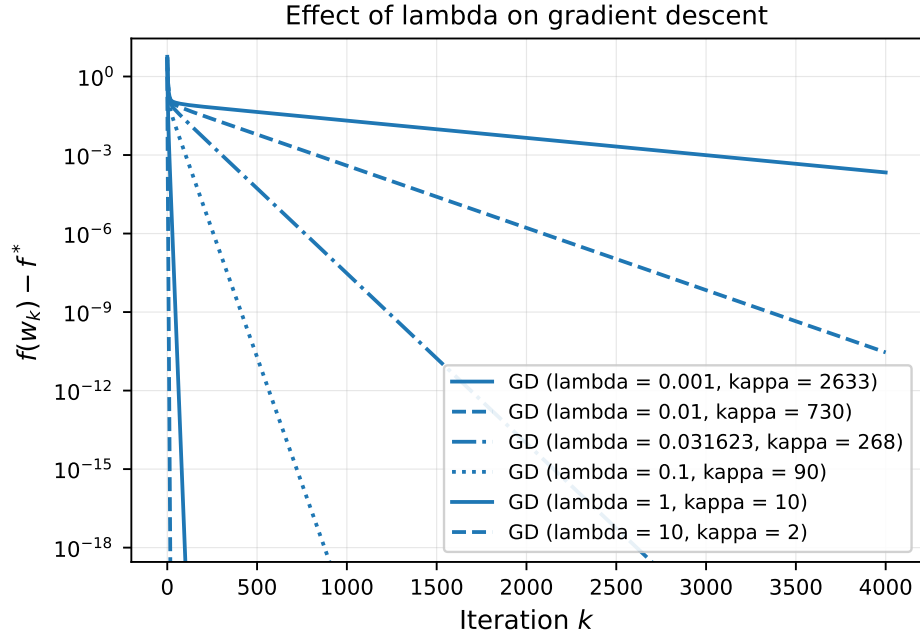
Tồn tại một khoảng λ mua được điều kiện gần như miễn phí. Đi từ $\lambda = 0,001$ lên $\lambda = 0,1$ làm RMSE trên tập kiểm tra xấu đi từ 3,4537 lên 3,4690, tức 0,44 phần trăm, trong khi κ giảm 29 lần và gradient descent đi từ chỗ không hội tụ nổi sau 4000 vòng xuống còn 257 vòng. Đường cong cross-validation phẳng trên khoảng đó, vì λ trong vùng này chưa đủ lớn để làm lệch nghiệm một cách đáng kể nhưng đã đủ lớn để nâng hần sần của phổ. Ra ngoài khoảng đó thì cái giá hiện ngay, với $\lambda = 1$ cho RMSE 3,7270 và $\lambda = 10$ cho 4,4578, tức xấu đi 29 phần trăm.

Giá trị $\lambda = 0,031623$ mà quy tắc một sai số chuẩn chọn ở chương 4 nằm đúng trong khoảng đó, với $\kappa = 267,5$ và RMSE 3,4563. Cũng cần ghi nhận rằng nghiệm không hiệu chỉnh cho RMSE 3,4614, tức nhỉnh hơn nghiệm Ridge được chọn; trên bộ dữ liệu này với n lớn hơn d hơn một nghìn lần, hiệu chỉnh gần như không mua thêm được gì về mặt thống kê, và toàn bộ lợi ích của nó nằm ở phía tối ưu hóa. Hình 11.1 vẽ các đường hội tụ tương ứng, còn hình 11.2 đặt κ và RMSE lên cùng một khung.

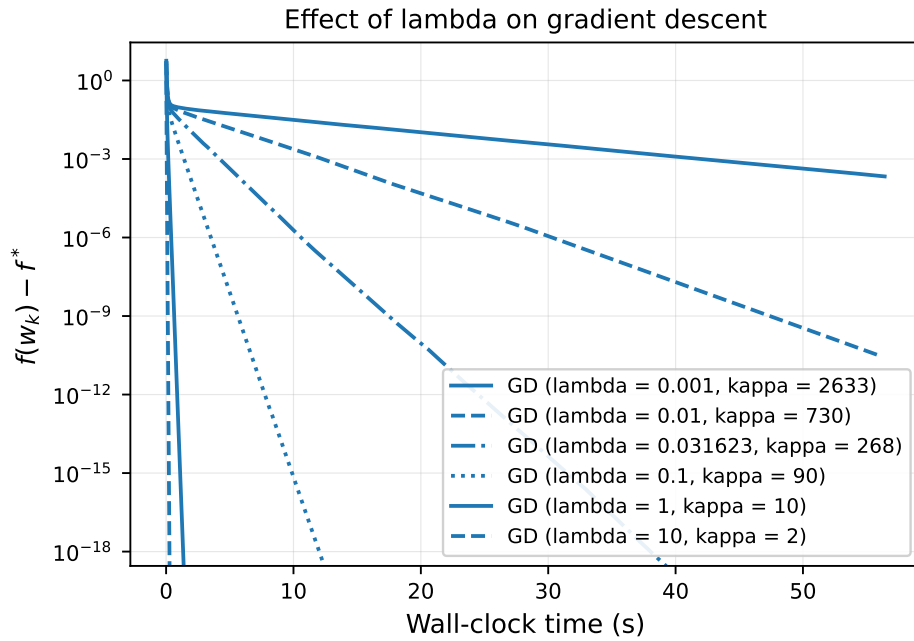
Bảng 11.1: Ảnh hưởng của λ lên số điều kiện, tốc độ hội tụ và chất lượng dự báo, $n = 200\,000$. Cột vòng lặp là số vòng cần để đạt $f - f^* < 10^{-6}$.

λ	μ	κ	GD	AGD	RMSE test
0,001	0,00345	2633,0	không đạt	209	3,4537
0,01	0,01245	730,4	2091	118	3,4541
0,031623	0,03407	267,5	766	75	3,4563
0,1	0,10245	89,6	257	48	3,4690
1	1,00245	10,1	29	17	3,7270
10	10,00245	1,9	5	6	4,4578

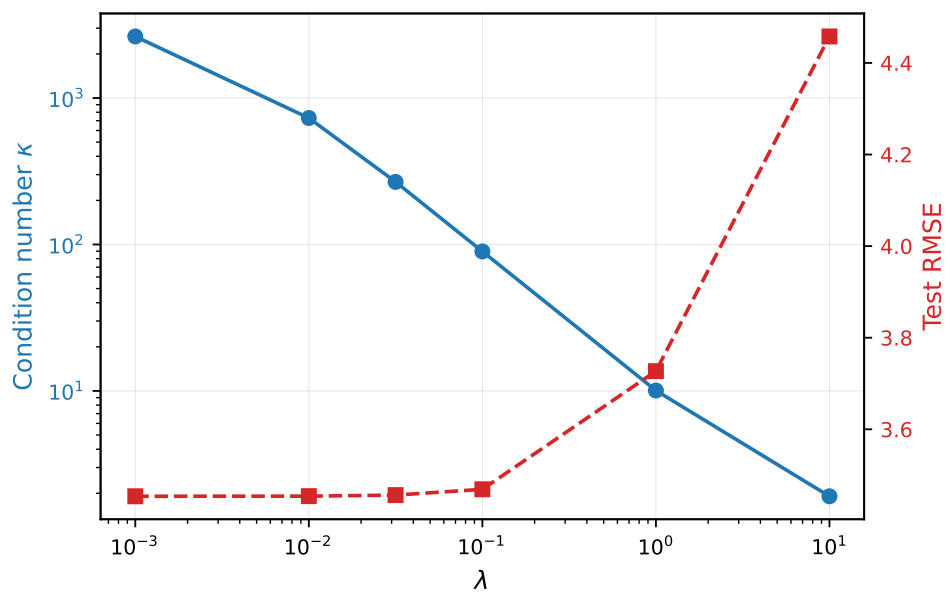
(a) theo số vòng lặp



(b) theo thời gian chạy



Hình 11.1: Gradient descent với các giá trị λ khác nhau, mỗi lần dùng bước $2/(L + \mu)$ của chính bài toán đó.



Hình 11.2: Số điều kiện và sai số dự báo vẽ cùng theo λ trên hai trục tung.

Kết luận

Báo cáo so sánh bốn thuật toán tối ưu hóa trên hai hàm mục tiêu dựng từ cùng dữ liệu Lending Club với $d = 116$: hàm mất mát Ridge với $\kappa = 267,5$, và hàm mất mát Huber ở chương 8 với κ đo tại w^* là 271,2. Ở cấu hình tốt nhất của từng phương pháp, tổng hợp ở chương 9, thứ hạng theo cả hai trục đo là Newton, gradient tăng tốc, gradient descent với backtracking, rồi gradient descent với bước cố định; SGD nhanh nhất trên mỗi đơn vị thời gian nhưng dừng ở sai số $6,0 \cdot 10^{-4}$. Thứ hạng đó không đổi khi chuyển sang hàm mục tiêu thứ hai.

Ba điều rút ra về việc chọn tham số. Với backtracking, ρ quan trọng hơn c rất nhiều, vì nó quyết định số lần thử mỗi vòng. Với SGD, bước phải đặt theo $1/L_B$ của kích thước lô chứ không theo $1/L$, và tốc độ giảm phải neo vào μ chứ không vào số vòng lặp mỗi epoch. Với gradient tăng tốc, β phải tính lại theo bước mà line search thực sự chọn, như chương 5 chỉ ra.

Năm chỗ thực nghiệm lệch khỏi phát biểu quen thuộc của lý thuyết hoặc khỏi dự đoán ban đầu. Ba chỗ đầu lệch vì cùng một lý do: cận lý thuyết là cận cho trường hợp xấu nhất trên một lớp bài toán, còn thực nghiệm chạy trên một bài toán cụ thể, một điểm khởi tạo cụ thể và một ngân sách hữu hạn. Bước $1,9/L$ nhanh gấp 2,3 lần bước tối ưu $2/(L + \mu)$, vì sai số ban đầu tại $w_0 = 0$ dồn 80,5 phần trăm vào các hướng trị riêng lớn. Backtracking thắng bước cố định trên cả hai trục, vì nó so với thương Rayleigh tại chỗ chứ không với L toàn cục. Và quy tắc giảm bước của SGD chỉ thắng bước hằng khi tốc độ giảm đặt đúng, với hai giá trị hợp lý cho hai kết luận ngược nhau.

Hai chỗ còn lại đến từ hàm mục tiêu Huber, và cả hai đều xoay quanh việc thông tin độ cong lấy ở đâu. Dừng lại Hessian hết miễn phí ngay khi hàm mục tiêu hết bậc hai: với bước đầy đủ, dãy lặp khóa vào một chu trình hai điểm và không hội tụ, còn thêm line search thì nó về đích sau 13 vòng. Ngược lại, dự đoán rằng bỏ tính bậc hai sẽ làm backtracking có việc chỉ đúng với hướng gradient, tốn 4,20 lần đánh giá hàm mỗi vòng, mà không đúng với hướng Newton, nơi line search nhận ngay bước đầy đủ ở mọi vòng vì bản thân hướng đó đã mang độ cong.

Kết luận cuối cùng về phạm vi áp dụng. Với $d = 116$ và n cỡ triệu, phương pháp bậc hai thắng áp đảo, vì chi phí dựng Hessian là $O(nd^2)$ với d^2 nhỏ, còn phần $O(d^3)$ chỉ chiếm 0,2 ms trong tổng 250 ms. Các phương pháp lặp bậc một chỉ thể hiện lợi thế khi d lớn tới mức không dựng nổi $X^T X$, hoặc khi dữ liệu không nạp hết vào bộ nhớ. Đối chiếu với scikit-learn ở chương 10 cho thấy nghiệm đóng tự cài và solver mặc định của thư viện mất cùng một thời gian, vì cả hai làm cùng một phép tính. Tính L và μ trước khi chọn bước tốn dưới một giây trên bài toán này, trong khi quét năm bước theo lối dò tay tốn 195 giây và vẫn không tìm ra bước gần tối ưu.

Tài liệu tham khảo

- [1] Stephen Boyd **and** Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.
- [2] Arthur E. Hoerl **and** Robert W. Kennard. “Ridge regression: biased estimation for nonorthogonal problems”. **in***Technometrics*: 12.1 (1970), **pages** 55–67.
- [3] Trevor Hastie, Robert Tibshirani **and** Jerome Friedman. *The Elements of Statistical Learning*. 2 **edition**. Springer, 2009.
- [4] Larry Armijo. “Minimization of functions having Lipschitz continuous first partial derivatives”. **in***Pacific Journal of Mathematics*: 16.1 (1966), **pages** 1–3.
- [5] Yurii Nesterov. *Lectures on Convex Optimization*. 2 **edition**. Springer, 2018.
- [6] Brendan O’Donoghue **and** Emmanuel Candès. “Adaptive restart for accelerated gradient schemes”. **in***Foundations of Computational Mathematics*: 15.3 (2015), **pages** 715–732.
- [7] Herbert Robbins **and** Sutton Monro. “A stochastic approximation method”. **in***The Annals of Mathematical Statistics*: 22.3 (1951), **pages** 400–407.
- [8] Léon Bottou, Frank E. Curtis **and** Jorge Nocedal. “Optimization methods for large-scale machine learning”. **in***SIAM Review*: 60.2 (2018), **pages** 223–311.
- [9] Jorge Nocedal **and** Stephen J. Wright. *Numerical Optimization*. 2 **edition**. Springer, 2006.
- [10] Peter J. Huber. “Robust estimation of a location parameter”. **in***The Annals of Mathematical Statistics*: 35.1 (1964), **pages** 73–101. doi: 10.1214/aoms/1177703732.
- [11] Fabian Pedregosa **and others**. “Scikit-learn: machine learning in Python”. **in***Journal of Machine Learning Research*: 12 (2011), **pages** 2825–2830.